

Single Core Optimization

HIGH PERFORMANCE AND CLOUD COMPUTING

09 March 2026

Indice

0.1	Optimization Preliminaries and Compiler Usage	3
0.1.1	Lecture Overview	3
0.1.2	Introduction to Optimization	4
0.1.3	First Steps Toward Optimization	5
0.1.4	The Optimization Process: A Step-by-Step Framework	6
0.1.5	The Compiler: Concepts and Usage	7
0.1.6	Four Pillars of Compiler-Friendly Code	9
0.1.7	Compiler Options and Optimization Levels	10
0.1.8	Memory Considerations	11
0.1.9	Memory Aliasing: A Major Optimization Blocker	12
0.2	Modern Architecture: Foundations for Efficient Code	14
0.2.1	Lecture Rationale	14
0.2.2	Outline	14
0.2.3	1. Facts and Historical Trends	14
0.2.4	2. The Memory Subsystem	17
0.2.5	3. Out-of-Order Execution, Pipelines, and Vectors	21
0.2.6	4. Historical Synthesis	23
0.2.7	5. Essential Vocabulary	24
0.3	Optimization: Basic Steps — Avoiding Trivial Inefficiencies	25
0.3.1	Lecture Context	25
0.3.2	Running Example: Grid-Based Neighbor Search	26
0.3.3	Optimization Principle 1: Avoid Expensive Function Calls	26
0.3.4	Optimization Principle 2: Hoisting Loop-Invariant Expressions	28

0.3.5	Optimization Principle 3: Clarify Variable Scope	29
0.3.6	Optimization Principle 4: Suggest Register Placement	30
0.3.7	Compiler Limitations and the Compiler’s Perspective	30
0.3.8	Floating-Point Arithmetic: Non-Associativity	30
0.3.9	Optimization Principle 5: Avoid Unnecessary Repeated Checks	32
0.3.10	Optimization Principle 6: Avoid Unnecessary Memory References	33
0.3.11	Summary of Basic Optimization Principles	35
0.4	The Execution Model: Heap and Stack	35
0.4.1	Lecture Context	35
0.4.2	The Virtual Memory Model	36
0.4.3	The Standard Execution Model	37
0.4.4	Stack and Heap: Fundamental Characteristics	38
0.4.5	Stack Mechanics in Detail	39
0.4.6	Stack Scope and Lifetime	40
0.4.7	Dynamic Allocation on the Stack: <code>alloca()</code>	41
0.4.8	Stack Size Management	42
0.4.9	Stack vs. Heap: Performance Comparison	42
0.4.10	Generating and Inspecting Assembly Output	44
0.4.11	Practical Considerations and Design Implications	45
0.5	Cache and Memory: Detailed Mechanisms	45
0.5.1	Cache–DRAM Mapping Strategies	45
0.5.2	Memory Access Patterns and Cache Behavior	47
0.5.3	The Memory Mountain	48
0.5.4	The Cache-Miss Signature	48
0.5.5	Achievable Bandwidth: A Rough Guide	49
0.5.6	Matrix Storage and Strided Access	49
0.5.7	Cache-Associativity Conflicts (Cache Resonance)	51
0.5.8	Enhancing Locality: Data Traversal Strategies	51
0.5.9	Hot and Cold Fields	53
0.6	Loop Characterization and Optimization	55
0.6.1	Arithmetic Intensity	55
0.6.2	$O(N)/O(N)$ Loops: Loop Fusion	55
0.6.3	$O(N^2)/O(N^2)$ Loops: Matrix–Vector Multiplication	56
0.6.4	$O(N^3)/O(N^2)$ Loops: Matrix–Matrix Multiplication	57

0.6.5	Loop Unrolling: Detailed Analysis	57
0.7	Prefetching	59
0.7.1	The Latency Hiding Problem	59
0.7.2	The Prefetching Principle	59
0.7.3	Prefetch Distance	59
0.7.4	Three Levels of Prefetching	59
0.7.5	Practical Example: Binary Search with Prefetching	60
0.7.6	Prefetching by Pre-loading	61
0.7.7	Best Practices Summary	61
0.8	Branches: Control Flow and Performance	62
0.8.1	Conditional Execution Mechanisms	62
0.8.2	Conditional Control Flow: Jump Instructions	62
0.8.3	Conditional Data Flow: The <code>cmov</code> Instruction	63
0.8.4	The Cost of Branch Misprediction	63
0.8.5	Strategies for Minimizing Branch Impact	64
0.8.6	Profile-Guided Optimization for Branches	65
0.8.7	A Word of Caution	66

0.1 Optimization Preliminaries and Compiler Usage

0.1.1 Lecture Overview

This lecture opens a series dedicated to **single-core code optimization**. It covers general concepts and preliminaries on what “optimizing” means in the context of High Performance Computing (HPC), along with practical guidance on compiler usage and memory-related considerations. Subsequent lectures in the series will address cache and loops, branches, pipelines, and memory in greater depth.

The broader optimization lecture arc follows this progression:

1. **First things first** — general concepts and compiler preliminaries (this lecture)
 2. **Cache & Loops** — cache-aware programming and loop optimization
 3. **Branches** — control flow and branch prediction
 4. **Pipelines** — instruction-level parallelism and pipelining
 5. **Memory** — memory hierarchy and access patterns
-

0.1.2 Introduction to Optimization

0.1.2.1 Defining “Optimization” High Performance Computing, by its very name, demands that one extracts the **maximum effectiveness** from code running on a given machine. Optimization is a key step in this process, yet both “optimizing” and “effectiveness” are terms that require careful definition. There is no such thing as a universally “optimal” code — optimality is always relative to a set of constraints and objectives.

0.1.2.2 The Multi-Dimensional Nature of Optimality Optimization in HPC is not a single-axis problem. A program may need to be optimal along several competing dimensions simultaneously:

Constraint Domain	Description
Time constraints	Minimizing time-to-solution (worst case, average case, or best case)
Memory constraints	Reducing both data and code memory footprint
I/O constraints	Carefully crafting disk, memory, and network I/O operations
Energy constraints	Minimizing energy-to-solution, increasingly important in large-scale systems
Robustness / Resilience	Ensuring correctness and fault tolerance in mission-critical applications

These dimensions often conflict: a code that is optimal for time-to-solution may consume excessive memory or energy, and vice versa. The art of optimization lies in navigating these trade-offs within the specific constraints of a given project.

0.1.2.3 Knuth’s Warning: Premature Optimization

“Premature optimization is the root of all evil.” — Donald Knuth

This celebrated dictum reminds us that optimization should never precede correctness or sound design. The priority order is:

1. **Correctness of the code** — a fast program that produces wrong results is worthless
2. **Data model** — the way data is structured fundamentally determines performance ceilings
3. **Algorithm selection** — a superior algorithm, even if poorly implemented, will almost always outperform a beautifully optimized but algorithmically inferior solution for sufficiently large problem sizes

0.1.2.4 From “Optimal” to “Improved” Code Rather than chasing an abstract notion of optimality, it is more productive to think in terms of **improved code**. Three guiding principles apply:

- **Correctness first:** never sacrifice correctness for speed. A faster code that yields incorrect results is not optimal in any sense.
 - **Readability and maintainability:** code is almost always revisited — by its author or by others. Clean, understandable, non-obfuscated code improves the quality of the entire development lifecycle.
 - **Collaborative usability:** others will likely need to read, understand, and extend the code. Clarity is a form of optimization in itself.
-

0.1.3 First Steps Toward Optimization

The path to improved code follows three broad stages:

1. **Clean code** — ensure the codebase is tidy, well-documented, and free of unnecessary complexity
2. **Re-design the fundamental architecture** — revisit the high-level structure of the program
3. **Improve specific aspects** — optimize workflow, memory footprint, resource usage, and time-to-solution

0.1.3.1 Writing Clean Code: The DRY Principle **DRY** (*Don't Repeat Yourself*) is a foundational software engineering principle. It prescribes:

- **No unnecessary code:** superfluous code increases the effort required for maintenance, debugging, updating, and extending functionality.
- **No duplicated code:** code duplication inflates **technical debt**, which accumulates from practices such as:
 - Copy-and-paste programming
 - Excessively agile (i.e., hasty) development approaches
 - Hard-coding values, quick-and-dirty fixes, *cargo-cult* programming, and general sloppiness

0.1.3.2 Readability and Maintainability **Readability** is a pillar of **maintainability**, which in turn ensures that software can be extended, upgraded, debugged, and fixed over time. Three levels of commenting discipline apply:

- **Baseline:** write comments in the code
- **Advanced:** write thorough, structured comments (preferably using tools such as Doxygen)
- **Expert-level:** avoid comments that are trivial, obvious, misleading, or redundant

A useful maxim: “*The code is the ultimate man page.*” Well-written code should be largely self-documenting.

0.1.3.3 Software Design Principles Good design for HPC codes involves multiple layers of consideration:

- **Consult the literature:** understand the mathematical and scientific aspects of the problem being solved
- **Account for hardware realities:** you are not working on a theoretical blackboard but on a digital, discrete system — equations must be carefully translated into computational form
- **Consider concurrency and parallelism** from the outset
- **Account for constraints:** memory, I/O, and energy constraints must inform the design

0.1.3.3.1 Hierarchical Design Structure A well-designed HPC application can be viewed at multiple levels of abstraction:

Level	Focus
Highest abstraction	Interacting multi-component systems; top-level data model
Sub-systems and modules	Communication strategies; modules as producers of complex tasks
Implementation details	Algorithm selection for each module; concurrency patterns
Logical structure	How modules interact at a structural level
Very low-level coding	Architecture-specific optimizations

0.1.3.3.2 Testing as Part of Design Testing is not an afterthought — it is an integral component of the design process:

- **Unit testing:** independently stress each individual unit (function, module) of the code
- **Integration testing:** stress the combined behavior of all units working together
- **System testing:** validate the complete system under realistic conditions

Two complementary quality assurance processes must be embedded:

- **Validation:** ensures that the code does *what it was meant to do* (all modules conform to design specifications)
- **Verification:** ensures that the code does *what it does correctly* (black-box testing against known test cases)

0.1.4 The Optimization Process: A Step-by-Step Framework

A systematic approach to optimization proceeds through the following stages:

1. **Correctness:** produce a program that delivers correct answers and behaves correctly under all conditions. The code must be clear, clean, concise, and well-documented.

2. **Algorithms and data structures:** adopt the best-suited algorithms and data structures. “Best-suited” is determined by the convolution of all applicable constraints (time-to-solution, energy-to-solution, memory, etc.).
3. **Compiler-friendly code:** ensure the source code is **optimizable by the compiler**. This requires a firm understanding of the compiler’s capabilities and limitations, as well as the target architecture. Understand the trade-off between portability and performance (factoring in human effort).
4. **Data-driven, task-based workflow:** design a workflow that is data-driven and task-based, which will almost certainly be parallel — in either distributed-memory, shared-memory paradigms, or both.
5. **Profiling:** profile the code under different conditions (workload, problem size, degree of parallelism, platform) to identify bottlenecks and inefficiencies.
6. **Hot-spot optimization:** apply specific optimization techniques to hot-spots in the code, removing optimization blockers and exposing intrinsic instruction-level and data-level parallelism.
7. **Iterate:** if needed, return to step 1 and repeat.

This is explicitly **not a linear process**. Modern architectures are so complex and evolve so rapidly that it can be difficult to perfectly model a code’s performance. Promising techniques may sometimes fail, and trial-and-error is an inherent part of the optimization cycle.

0.1.5 The Compiler: Concepts and Usage

0.1.5.1 What Is a Compiler? A **compiler** is a program that translates a program written in a **source language** into an equivalent program in a **target language**, while also detecting and signaling errors (primarily semantic errors and a subset of other error types). If the target language is executable by a machine, the resulting program can be directly executed to process inputs and produce outputs.

0.1.5.2 Compiler vs. Interpreter An **interpreter** is a different kind of language processor that directly executes instructions written in a source language, without producing a separate executable.

Property	Compiled Languages	Interpreted Languages
Execution speed	Generally much faster	Generally slower
Error analysis	Limited to compile-time	Enhanced runtime error detection
Portability	Architecture-dependent binaries	Higher portability

0.1.5.3 The Compilation Pipeline In practice, a compiler is only one stage in a multi-step compilation pipeline:

1. **Preprocessor**: processes the source code by including headers, expanding macros, and handling conditional compilation directives
2. **Front-end (compiler proper)**: translates the preprocessed source (C, C++, Fortran, etc.) into an intermediate representation or directly into assembly code
3. **Assembler**: converts assembly code into **relocatable machine code** (object code) for each compilation unit
4. **Linker**: resolves memory addresses across different sections of code and connects references to external libraries

0.1.5.4 Compilation Example Consider a minimal C program:

```
int main(void) {  
    int a = 1;  
    int b = 2;  
    return a + b;  
}
```

The **compiler** translates this into x86_64 assembly:

```
mov DWORD PTR -8[rbp], 1  
mov DWORD PTR -4[rbp], 2  
mov edx, DWORD PTR -8[rbp]  
mov eax, DWORD PTR -4[rbp]  
add eax, edx  
ret
```

The **assembler** then converts the assembly into machine code (shown as hexadecimal via **objdump**):

```
c7 45 f8 01 00 00 00  
c7 45 fc 02 00 00 00  
8b 55 f8  
8b 45 fc  
01 d0  
c3
```

The entire pipeline is invoked with a single command:

```
cc -o example example.c
```

The resulting binary can be inspected with **objdump -d example**.

0.1.5.5 The Compiler as Optimizer Beyond translation, modern compilers perform **sophisticated analysis** of source code to produce target code that is highly optimized for a given architecture. The general guideline is:

“Optimization” largely means “let the compiler squeeze the maximum from your code.”

Compilers are remarkably capable and possess deep knowledge of the hardware they target. The programmer’s role is to write code that enables the compiler to do its best work.

0.1.6 Four Pillars of Compiler-Friendly Code

0.1.6.1 1. Write Non-Obfuscated Code

- **Avoid memory aliasing** (discussed in detail below)
- Make it clear what each variable is used for and when
- Take care of loop structures
- Keep conditional branches under control

0.1.6.2 2. Design a Good Data Structure Layout

- Be **cache-friendly** (but *cache-oblivious* — i.e., design patterns that work well across different cache sizes)
- Follow the principle: *“what is used together, stays together”*
- Be **NUMA-conscious** (Non-Uniform Memory Access)
- Avoid **false sharing** in multi-threaded environments

0.1.6.3 3. Design a Good Workflow

- A well-structured workflow allows the compiler to optimize branches and memory access patterns effectively
- **Prefetching** will work better with predictable access patterns
- A clean workflow makes it easier to introduce multi-threading

0.1.6.4 4. Exploit Modern Architecture Features

- Let the compiler exploit **pipelining** through operation ordering and loop unrolling
 - Let the compiler exploit the **vectorization** capabilities (SIMD) of modern CPUs
 - Think **task-based** and **data-driven**
-

0.1.7 Compiler Options and Optimization Levels

0.1.7.1 Basic Compiler Invocations

Command	Purpose
<code>cc source.c -o executable</code>	Basic compilation
<code>cc source.c -g -o executable</code>	Compilation with debugging information
<code>cc -On source.c -o executable</code>	Compilation with optimization level n (typically 1, 2, or 3)

Widely used, high-quality C/C++ compilers include **GCC** (`gcc`), **Clang** (`clang`), **PGI** (`pgi`), and **Intel** (`icc`). The Godbolt Compiler Explorer is an excellent tool for interactively examining compiler output.

0.1.7.2 Optimization Levels (-On) The flag `-O3` enables aggressive optimizations, but it is **not always the best choice**. For instance:

- Aggressive optimizations may generate **larger code**, which on architectures with smaller caches may actually run **slower**
- In such cases, `-Os` (optimize for size) may yield surprising performance improvements

Modern compilers also support **function-level optimization attributes**, allowing fine-grained control. In GCC:

```
__attribute__((__option__("...")))  
__attribute__((optimize(n)))
```

0.1.7.3 Compiling for a Specific CPU Model (-march=native) By default, the compiler generates **portable code** — code that runs on any CPU within a given architecture class (e.g., x86_64, ARM, POWER9). However, specific CPU models support specialized instruction set extensions that are not universally available.

Using architecture-specific flags instructs the compiler to optimize for the exact CPU it is running on:

Compiler	Flag
GCC / Clang	<code>-march=native -mtune=native</code>
Intel ICC	<code>-xHost</code>

This typically produces more performant code, at the cost of **reduced portability** — the resulting binary may not run on different CPU models.

0.1.7.4 Profile-Guided Optimization (PGO) **Profile-guided optimization** allows the compiler to instrument the code during a first compilation pass, collect runtime execution data, and then use that data in a subsequent compilation to make more informed optimization decisions. This is especially effective when the code contains many branches.

For GCC, two workflows are available:

Branch-specific PGO:

```
gcc -fprofile-arcs source.c -o instrumented
./instrumented           # run with representative input
gcc -fbranch-probabilities source.c -o optimized
```

General PGO:

```
gcc -fprofile-generate source.c -o instrumented
./instrumented           # run with representative input
gcc -fprofile-use source.c -o optimized
```

The general PGO approach also enables `-fprofile-values` and `-freorder-functions` for additional optimizations.

0.1.8 Memory Considerations

0.1.8.1 Memory Allocation Efficient memory allocation is a prerequisite for optimization:

- **Allocate contiguous memory** whenever possible
- **Re-use memory efficiently**, avoiding fragmentation

0.1.8.2 C Storage Classes

Storage Class	Description
<code>extern</code>	Global variables; they persist for the entire program lifetime
<code>auto</code>	Local variables allocated on the stack within a limited scope, then destroyed; must be explicitly initialized
<code>register</code>	Suggests to the compiler that the variable should be placed directly in a CPU register for faster access

0.1.8.3 C Variable Qualifiers

Qualifier	Description
<code>const</code>	Indicates the variable will not be modified within the current scope
<code>volatile</code>	Indicates the variable may be accessed or modified from outside the program (e.g., by hardware or another thread)
<code>restrict</code>	Guarantees that the memory address is accessed only through the specified pointer — no other pointer aliases this memory

0.1.9 Memory Aliasing: A Major Optimization Blocker

0.1.9.1 The Problem Memory aliasing occurs when two or more pointer variables reference the same memory location. It is one of the most significant optimization blockers in languages that permit pointer arithmetic, such as C and C++.

0.1.9.2 Illustrative Example Consider the following two functions:

```
void func1(int *a, int *b) {  
    *a += *b;  
    *a += *b;  
}
```

```
void func2(int *a, int *b) {  
    *a += 2 * (*b);  
}
```

At first glance, `func2` appears to be an optimized version of `func1` — it has fewer memory references and should produce the same result. However, this equivalence **breaks** when `a == b` (i.e., both pointers reference the same memory location).

If `*a = 1` and `a == b`:

- **func1**: after `*a += *b`, both contain 2; after the second `*a += *b`, both contain 4
- **func2**: `*a += 2 * (*b)` yields 3

Because the compiler cannot, in general, prove that `a` and `b` do not alias, it must conservatively assume they might, thereby **preventing the optimization** from `func1` to `func2`.

0.1.9.3 The restrict Qualifier The **restrict** keyword in C99 and later tells the compiler that a pointer is the **sole means of accessing** the memory it points to — no other pointer will alias it. This allows the compiler to perform optimizations that would otherwise be unsafe.

Without **restrict**:

```
void my_function(double *a, double *b, int n) {
    for (int i = 0; i < n; i++)
        a[i] = s * b[i - 1];
}
```

The compiler cannot optimize memory accesses because **a** and **b** might overlap.

With **restrict**:

```
void my_function(double *restrict a, double *restrict b, int n) {
    for (int i = 0; i < n; i++)
        a[i] = s * b[i - 1];
}
```

Now the compiler knows the memory regions referenced by **a** and **b** will never overlap, enabling aggressive optimization of memory accesses (e.g., avoiding redundant reads).

0.1.9.4 Compiler Behavior and Aliasing Modern compilers are sophisticated enough to handle some aliasing cases automatically. When analyzing a loop with potentially aliasing pointers, the compiler may generate **two separate code paths**:

1. A **safe loop** for the case where memory regions overlap
2. An **optimized loop** for the case where they do not

A runtime check determines which path to execute. The optimized path closely resembles the code generated when the programmer explicitly uses **restrict**.

0.1.9.5 Additional Compiler Flags for Aliasing Beyond the **restrict** keyword, GCC provides compiler flags to control aliasing assumptions:

- **-fstrict-aliasing**: enables the compiler to assume strict aliasing rules (type-based alias analysis)
- **-Wstrict-aliasing**: issues warnings when code might violate strict aliasing rules

It is worth noting that **Fortran** formally forbids aliasing, which is one reason why Fortran compilers can sometimes produce faster executables than C compilers without any special programmer intervention.

0.2 Modern Architecture: Foundations for Efficient Code

0.2.1 Lecture Rationale

This lecture presents, in a highly condensed form, the fundamental traits of **single-core modern architecture**, traced through their historical development. The central thesis is that understanding the hardware is not merely helpful but a **prerequisite** for writing efficient code. Every optimization technique discussed in subsequent lectures is rooted in the architectural features surveyed here.

0.2.2 Outline

The lecture is organized into seven thematic blocks:

1. Facts and historical trends
2. The memory subsystem
3. Out-of-Order execution, Instruction-Level Parallelism, Pipelines, and Vectors
4. Branches
5. Many cores
6. Many nodes
7. Heterogeneous devices

Topics 4–7 are introduced briefly here and developed in dedicated lectures (branches, OpenMP, MPI, and GPU/Advanced HPC, respectively).

0.2.3 1. Facts and Historical Trends

0.2.3.1 The Von Neumann Architecture vs. Today’s Reality The **Von Neumann architecture** remains the foundational conceptual model taught in computer science. Its defining characteristics are:

- **Single processing unit:** only one CPU performs all computation
- **Sequential execution:** one instruction is executed at a time
- **Flat memory model:** every memory location has the same access cost, and memory access cost is comparable to the cost of executing an arithmetic operation

A Von Neumann machine consists of:

Component	Role
Control Unit	Fetches and sequences instructions
ALU (Arithmetic/Logic Unit)	Performs arithmetic and logical operations
Registers (PC, IP, SP, ...)	Hold addresses and intermediate values
Memory	Stores both data and instructions uniformly
Input / Output	Interfaces with the external world

Modern processors violate virtually every simplifying assumption of this model:

- **Many processing units** coexist on a single chip
- **Many instructions** can be executed simultaneously (superscalarity, pipelining)
- **Many data elements** can be processed at once (SIMD vectorization)
- Instructions are internally decomposed into simpler **micro-operations** (μops) that are pipelined; different instructions may have vastly different execution costs
- **Memory is strongly non-flat**: a deep hierarchy exists, and access latency varies by orders of magnitude depending on the data's location (register, L1 cache, L2 cache, L3 cache, DRAM)

As a concrete example, a 6-core Intel Xeon E5600 exhibits a memory hierarchy where accessing RAM can be **hundreds of times more costly** (measured in CPU cycles) than performing a μop or reading an internal register.

0.2.3.2 Moore's Law **Moore's law** — an empirical observation rather than a physical law — states that the number of transistors per unit area on an integrated circuit **doubles** approximately every 18–24 months, at constant manufacturing cost. For over five decades, this trend held remarkably steady.

The practical consequences were profound:

- More transistors enabled **more complex circuits** with enhanced capabilities
- Higher transistor densities allowed **higher clock frequencies**, making code execution faster without any programmer effort

This period is often called the “**free lunch era**”: performance improvements came essentially for free from the hardware side, without requiring changes to software.

However, every exponential growth encounters limits. The free lunch era came to an end in the mid-2000s.

0.2.3.3 The Dennard Scaling **Dennard's scaling** (Robert Dennard, 1974) described the principle of **constant-field scaling**. If every linear dimension of a transistor is shrunk by a factor $\kappa > 1$ (so that the new length $\ell' = \ell/\kappa$), and supply and threshold voltages are also scaled by $1/\kappa$, while doping concentrations are adjusted appropriately, then:

- **Electric fields remain roughly constant**, since $E \sim V/\ell$ and both V and ℓ scale by $1/\kappa$
- **Capacitance per gate** scales as $C \propto \varepsilon \cdot \ell/t \sim 1/\kappa$
- **Delay** scales down: $\tau \sim RC \propto 1/\kappa$, so **frequency** goes up: $f \propto \kappa$
- **Dynamic power per gate**: $P_{\text{dyn}} \propto CV^2f \propto (1/\kappa)(1/\kappa^2)(\kappa) = 1/\kappa^2$
- **Transistor density** rises as κ^2

Multiplying density by per-gate power yields **power density constant**. The motto: *smaller and faster at constant heat*.

0.2.3.4 The Breaking of Dennard’s Scaling Dennard’s scaling broke down in the mid-2000s due to multiple converging physical limitations:

1. **Voltage stopped scaling.** The threshold voltage V_{th} cannot be reduced indefinitely. The subthreshold slope at room temperature bottoms out near ~ 60 mV/decade for an ideal MOSFET. With supply voltage V_{DD} stuck at approximately 1 V, the key energy term CV_{DD}^2 ceased shrinking.
2. **Leakage currents exploded.** As channels thinned and gate oxides approached thicknesses of only a few atomic layers, gate tunneling current and **Drain-Induced Barrier Lowering** (DIBL) caused exponential growth in leakage. Static power ($P_{static} \propto I_{leakage} \times V$) became a first-order contribution, potentially amounting to $\sim 25\%$ of total power consumption.
3. **Velocity saturation and mobility limits.** At nanometer-scale electric fields, charge carriers hit velocity saturation; the ideal $1/\kappa$ delay improvement is no longer achievable.
4. **Interconnect bottleneck.** Wires did not follow device scaling: RC delays worsened as cross-sections shrank, and resistivity effectively increased due to surface and grain-boundary scattering. Moving charge across the chip became slower and costlier than switching a gate.
5. **Thermal reality (the power wall).** With V_{DD} pinned but clock frequencies still rising, power density increased. Chips could no longer dissipate the generated heat — they would melt at higher frequencies. A typical CPU went from ~ 2 W to ~ 100 W, approaching the limit of air-cooling capacity.

0.2.3.5 The Power Wall and Mitigation Strategies The **power wall** — the inability to further increase clock frequency without exceeding thermal limits — forced a paradigm shift in processor design. Primary strategies to cope with the power wall include:

1. **Dark silicon:** turning off inactive circuits to reduce power consumption
2. **Dynamic Voltage and Frequency Scaling (DVFS):** downscaling voltage and frequency for both cores and DRAM
3. **Thermal Design Power (TDP):** designing for the typical case rather than the worst case
4. **Turbo Boost / Overclocking:** allowing short bursts of higher frequency on a fraction of the chip

0.2.3.6 Energy Cost of Operations Understanding the **relative energy cost** of different operations is essential for optimization-aware programming. For modern technology nodes (e.g., TSMC 7 nm):

- **Integer arithmetic** operations are the cheapest
- **Floating-point arithmetic** costs more, with FP multiply and FP add differing significantly
- **SRAM (cache) access** is more expensive than arithmetic but far cheaper than DRAM access
- **DRAM access** is the most energy-expensive common operation, often costing orders of magnitude more than arithmetic

This energy hierarchy reinforces the importance of **data locality**: keeping data close to the compute units (in cache rather than in DRAM) saves both time and energy.

0.2.3.7 The Clock Frequency–Performance Relationship Historically, single-core performance tracked clock frequency growth closely. When frequency scaling stalled around 2004–2006, single-core performance gains also flattened dramatically. The response was the shift toward **multi-core** architectures — packing more (simpler) cores onto a chip rather than making a single core faster. This transition marked the end of the free lunch era and placed the burden of performance squarely on the **software side** (parallel programming).

0.2.4 2. The Memory Subsystem

0.2.4.1 The CPU–Memory Performance Gap Starting in the early 1990s, CPU performance and memory (DRAM) performance diverged sharply:

Component	Historical improvement rate
CPU performance	~50% per year (until ~2004), then ~25% per year
DRAM latency	~7% per year
DRAM bandwidth	~20% per year

The consequence is stark: the CPU may spend **more time waiting for data from RAM** than actually executing operations. This is known as the **memory wall**.

0.2.4.2 Latency vs. Bandwidth Two fundamental metrics characterize memory performance:

Metric	Definition
Latency (Response Time)	The time between the start and the completion of a single data access event
Bandwidth (Throughput)	The total amount of data moved (or work done) per unit of time

In the context of execution:

- **Execution latency:** the total time to get a result from a single instruction (e.g., ~400 ps for a simple integer operation)
- **Execution bandwidth:** the number of instructions retired per second (measured in GIPS — giga-instructions per second)

In the context of memory access:

- **Memory latency:** the time the CPU must wait for a single data element to arrive (e.g., ~50 ns for DRAM, a few ns for SRAM/cache)
- **Memory bandwidth:** the rate at which data can be continuously streamed from memory (measured in GB/s)

The critical gap is in **latency**: there is a factor of $\sim 100\text{--}400\times$ between CPU operation latency ($\sim 0.25\text{--}1$ ns) and DRAM access latency ($\sim 50\text{--}100$ ns). While DRAM frequencies have increased (up to 3–4 GHz in modern DDR5), this alleviates the bandwidth problem only partially — the **latency gap** remains the dominant bottleneck.

0.2.4.3 The Principle of Data Locality Programs exhibit characteristic **data access patterns**: they tend to re-use data and instructions that have been accessed recently. Instruction re-use is typically stronger than data re-use. This empirical observation is formalized as the **principle of locality**, which comes in two forms:

- **Temporal locality**: if a memory address is referenced, it is likely to be referenced again in the near future
- **Spatial locality**: if a memory address is referenced, nearby addresses (in the address space) are likely to be referenced as well

The principle of locality is the theoretical foundation for the entire cache hierarchy.

0.2.4.4 The Memory Hierarchy The conflicting demands for **large** and **fast** memory cannot be simultaneously satisfied by a single technology. However, the principle of locality combined with the fact that smaller memories can be manufactured to be much faster (at acceptable economic and energetic cost) leads to a natural solution: a **hierarchy of memories** with different speeds, sizes, and costs.

The hierarchy, from fastest/smallest to slowest/largest:

Level	Typical Size	Typical Latency	Location
Registers	$\sim$ hundreds of bytes	< 1 ns	Inside the core
L1 Cache	$\sim 32\text{--}64$ KB (data + instructions)	$\sim 1\text{--}2$ ns ($\sim 2\text{--}4$ cycles)	Inside each core
L2 Cache	~ 256 KB – 1 MB	$\sim 4\text{--}10$ ns (~ 10 cycles)	Inside the core or shared
L3 Cache (LLC)	$\sim 8\text{--}64$ MB	$\sim 10\text{--}40$ ns ($\sim 30\text{--}70$ cycles)	Shared across cores, on-chip
DRAM (Main Memory)	$\sim 16\text{--}512$ GB	$\sim 50\text{--}100$ ns ($\sim 100+$ cycles)	Off-chip, on motherboard
Disk / SSD	TB-scale	$\sim 10^4\text{--}10^7$ ns	External storage

In HPC and server-grade platforms, additional levels may include **HBM** (High-Bandwidth Memory) and network-attached storage.

0.2.4.5 Cache Memory The **cache** is a small, fast memory placed extremely close to (or inside) the CPU core. Its purpose is to hold a subset of the data and instructions that the CPU is currently working with, thereby reducing the average memory access time.

The cache itself has an internal hierarchy:

- **L1 (Level 1)**: inside each core, typically split into separate instruction cache (I-cache) and data cache (D-cache), each ~32 KB
- **L2 (Level 2)**: also inside the core or shared between a small number of cores, typically ~256 KB to 1 MB
- **L3 (Level 3)**: inside the CPU package, shared by all cores, typically several MB to tens of MB

0.2.4.6 Quantifying the Cache Impact The **effective memory access time** is a weighted average that depends on the hit rate at each cache level. For a simplified single-level cache model:

$$T_{\text{eff}} = T_{L1} + \text{MissRate}_{L1} \times T_{\text{RAM}}$$

Assuming $T_{L1} = 2$ cycles and $T_{\text{RAM}} = 100$ cycles:

L1 Hit Rate	Effective Latency
100%	2 cycles
99%	3 cycles
97%	5 cycles

Adding a second cache level mitigates the severity of L1 misses:

$$T_{\text{eff}} = T_{L1} + \text{MissRate}_{L1} \times (T_{L2} + \text{MissRate}_{L2} \times T_{\text{RAM}})$$

Assuming $T_{L2} = 10$ cycles:

L1 Hit Rate	L2 Hit Rate	Effective Latency
100%	—	2 cycles
99%	100%	2.1 cycles
97%	100%	2.3 cycles
90%	97%	3.3 cycles

These numbers demonstrate that even small reductions in cache hit rate can **dramatically** increase effective memory access time, underscoring the importance of writing cache-friendly code.

0.2.4.7 Cache Misses: The Three C's Cache misses are categorized into three types, often called the **three C's**:

Miss Type	Cause
Compulsory misses	Unavoidable misses when data are accessed for the first time (also called <i>cold misses</i>)
Capacity misses	The working set exceeds the cache size; too much data is accessed between successive uses of the same element

Miss Type	Cause
Conflict misses	Multiple data items map to the same cache line(s) due to the cache's associativity structure (<i>cache thrashing</i>)

0.2.4.8 Strategies for Cache-Friendly Code The three corresponding strategies to improve cache utilization are often summarized as the **three R's**:

1. **Rearrange** (code and data): design data layouts and access patterns to improve both temporal and spatial locality
2. **Reduce** (size): use smaller data types where possible, access data in smaller chunks, and minimize the instruction footprint
3. **Reuse** (cache lines): increase temporal and spatial locality by keeping resident data in cache for as many operations as possible before evicting it

0.2.4.9 The Memory Mountain The interaction between cache hierarchy and data access patterns produces a characteristic performance landscape known as the **memory mountain**. This three-dimensional surface plots achieved memory throughput as a function of:

- **Working set size** (which determines which cache level is active)
- **Stride** (which determines the degree of spatial locality exploited)

Peaks correspond to small working sets with unit stride (full spatial and temporal locality); valleys correspond to large working sets with large strides (no locality, direct DRAM access).

0.2.4.10 The Cache-DRAM Mapping Problem A fundamental design challenge arises from the size disparity: a typical L1 data cache holds $\sim 10^4$ bytes (32 KB), while main memory holds $\sim 10^{10}$ bytes. The question of **how to map RAM addresses into cache locations** involves several sub-problems:

- **Placement**: where in the cache should a given memory block be stored?
- **Replacement**: if the target cache location is already occupied, which block should be evicted?
- **Identification**: how does the hardware quickly determine whether a block is in the cache?

These topics (direct-mapped, set-associative, and fully-associative caches) are developed in the dedicated cache lecture.

0.2.5 3. Out-of-Order Execution, Pipelines, and Vectors

0.2.5.1 Core Microarchitecture Overview A modern CPU core (e.g., Intel’s 6th-generation Skylake microarchitecture) is organized into two major sections:

- **Front-End:** responsible for **fetching** instructions and the data they operate on from the instruction and data caches, **decoding** instructions into μ ops, **predicting branches**, and **dispatching** μ ops to the execution units
- **Back-End:** responsible for **executing** the μ ops across multiple functional units and **writing back** results to memory locations; it also orchestrates **out-of-order** execution depending on instruction and data dependencies

Modern cores expose **multiple execution ports**, each specialized for different classes of operations (integer ALU, load/store, floating-point, FMA — Fused Multiply-Add, shift, LEA — Load Effective Address, etc.). This is the basis of **superscalar** capacity.

0.2.5.2 Instruction Pipelining

0.2.5.2.1 From Atomic to Pipelined Execution In early processors (before the mid-1980s), each instruction was executed as an atomic, indivisible operation. However, instruction execution naturally decomposes into several **independent stages**:

1. **Fetch (F):** retrieve the instruction from memory or the instruction cache
2. **Decode (D):** interpret the instruction and determine the required operands
3. **Execute (E):** perform the actual computation
4. **Writeback (W):** store the result in a register or memory

0.2.5.2.2 Non-Pipelined Execution If all four stages take approximately equal time (~100 ps each), a single instruction has a total **latency** of ~400 ps, yielding a throughput of:

$$\text{Throughput} = \frac{1}{400 \text{ ps}} = 2.5 \text{ GIPS}$$

0.2.5.2.3 Pipelined Execution By detaching the stages into independent hardware units (analogous to an assembly line), different instructions can occupy different stages simultaneously:

Cycle	Stage 1 (F)	Stage 2 (D)	Stage 3 (E)	Stage 4 (W)
1	I	—	—	—
2	I	I	—	—
3	I	I	I	—
4	I	I	I	I

The **latency** of a single instruction remains ~400 ps (it still passes through all four stages), but the **throughput** increases dramatically: once the pipeline is full, one instruction is **retired** (completed) every stage-delay, not every full-instruction-delay.

If stage delays are non-uniform (e.g., $F = 70$ ps, $D = 80$ ps, $E = 220$ ps, $W = 30$ ps), the throughput is limited by the **longest stage**:

$$\text{Throughput} = \frac{1}{\max(T_F, T_D, T_E, T_W)} = \frac{1}{220 \text{ ps}} \approx 4.5 \text{ GIPS}$$

0.2.5.2.4 Superpipelining The optimal pipeline has **all stages of equal duration**. This can be achieved by further subdividing the longer stages into multiple shorter sub-stages. If each sub-stage takes ~ 50 ps, then:

$$\text{Throughput} = \frac{1}{50 \text{ ps}} = 20 \text{ GIPS}$$

This technique is called **superpipelining**. Modern CPUs typically have 10–20 pipeline stages. The cost of deeper pipelines is a slightly increased total latency (due to additional logic between stages) and greater sensitivity to pipeline **stalls** (caused by hazards).

Key insight: pipelining is about increasing **throughput** (instructions retired per unit time), not reducing **latency** (time for a single instruction to complete). The goal is to reduce the **Cycles Per Instruction (CPI)** toward 1.

0.2.5.3 Superscalar Execution A **superscalar processor** replicates functional units and adds dispatch logic to issue and execute **more than one instruction per clock cycle**. While pipelining aims for $\text{CPI} = 1$, superscalarity breaks the $\text{CPI} = 1$ ceiling, achieving $\text{CPI} < 1$, or equivalently, **Instructions Per Cycle (IPC)** > 1 .

The key architectural feature is the presence of **multiple independent execution units** (ALUs, FPUs, load/store units, branch units) fed by a front-end capable of fetching, decoding, and dispatching $N > 1$ instructions per cycle. In an ideal superscalar processor with issue width W , the IPC can reach up to W .

0.2.5.4 Out-of-Order (OoO) Execution **Out-of-order execution** decouples the program order from the execution order. Data-independent instructions are reshuffled so that an instruction executes **as soon as its inputs are available** and a functional unit is free, regardless of its position in the original program sequence.

OoO execution is a form of **spatial reordering**: it determines which instructions execute across which functional units. It relies on **register renaming** to eliminate false dependencies (**Write-After-Read** and **Write-After-Write** hazards), exposing more instruction-level parallelism (ILP) to the hardware scheduler.

The historical foundation for OoO execution is **Tomasulo’s algorithm** (1967), originally developed for the IBM System/360 Model 91.

0.2.5.5 SIMD Vector Processing **Vector registers** are specialized, wide memory storage locations within the CPU designed to hold **multiple data elements** simultaneously. Unlike scalar registers, which process a single data element per instruction, vector registers allow the CPU to apply

one operation to all contained data elements at once, following the **Single Instruction, Multiple Data (SIMD)** paradigm.

SIMD introduces **Data-Level Parallelism (DLP)**, complementing the Instruction-Level Parallelism (ILP) provided by pipelining and superscalarity.

The three main SIMD instruction set families are:

ISA Family	Architecture	Notable Extensions
x86_64 SIMD	Intel / AMD	SSE, AVX, AVX-2, AVX-512
ARM SVE / NEON	ARM	Scalable Vector Extension, NEON
RISC-V Vector	RISC-V	RVV (RISC-V Vector Extension)

Not all loops are vectorizable; vectorizability depends on data dependencies, loop structure, memory access patterns, and other factors.

0.2.5.6 Summary of Core-Level Parallelism Technologies

Technology	Issue Addressed	Improvement
Pipelining	Sequential execution bottleneck	Throughput disentangled from latency; $IPC \rightarrow 1$
Superscalarity	Limit of $IPC = 1$	IPC up to W (the issue width)
Register Renaming	False dependencies (WAR / WAW hazards)	Exposes true ILP
Out-of-Order Execution	In-order stalls due to data dependencies	ILP up to W ; fills pipeline bubbles
SMT (Simultaneous Multi-Threading)	Single-thread ILP limits	Fills pipeline slots with instructions from another thread

0.2.6 4. Historical Synthesis

The evolution of computer architecture can be summarized across several major epochs:

Era	Key Developments
1950s–1970s	Simpler architectures; memory latency comparable to CPU speed; frequency increases drove performance gains; Tomasulo’s algorithm laid foundations for OoO
1980s–1990s	Moore’s law fueled more transistors $\Rightarrow$ superscalarity (multiple functional units), OoO execution, ILP — but ILP became power-hungry

Era	Key Developments
1990s–2000s	The memory wall era — CPU outpaced DRAM latency by orders of magnitude; deep cache hierarchies, TLBs, hardware prefetchers, branch predictors, and SIMD emerged
Mid-2000s	The power wall and the shift to multi-core ; many simpler cores replaced the single high-frequency monster; Thread-Level Parallelism (TLP) introduced new challenges: cache coherency, memory consistency models, NUMA
Last ~20 years	The Cambrian epoch — wider vectors, heterogeneous devices (GPUs, TPUs), SIMD on CPUs, SIMT on GPUs, HBM, high-speed interconnects (NVLink, Infinity Fabric), DPUs
Future	Proximity computation (compute near/in memory), neuromorphic devices, domain-specific architectures and ISAs

0.2.7 5. Essential Vocabulary

Term	Definition
Functional Units	Hardware blocks inside a CPU core that execute specific classes of operations (integer/FP arithmetic, memory load/store, branch control, vector/SIMD operations)
Out-of-Order Execution	A CPU strategy that allows μ ops to execute as soon as their inputs are ready, rather than strictly following program order
ILP / TLP	Instruction-Level Parallelism: multiple instructions from a single thread executing simultaneously; Thread-Level Parallelism: multiple threads executing concurrently
Cache	Small, fast memory placed close to or inside CPU cores to reduce average memory access time
TLB (Translation Lookaside Buffer)	A cache for virtual-to-physical address translations (covered in detail in the memory lecture)
Pipelines	The decomposition of instruction execution into sequential stages, allowing temporal overlap of different instructions

Term	Definition
Prefetcher	Hardware logic that fetches data and instructions from DRAM before they are needed, based on access pattern predictions
SIMD / SIMT	Single Instruction, Multiple Data (CPU vectors) / Single Instruction, Multiple Threads (GPU execution model)
Branch Predictor	Hardware logic that heuristically predicts whether a conditional branch will evaluate to true or false, to keep pipelines filled
NUMA	Non-Uniform Memory Access : a memory architecture where access latency depends on the relative location of the accessing core and the memory bank
Cache Coherency	The protocol that ensures consistent propagation of data modifications across the private caches of different cores
Vector Registers	Wide CPU registers that hold multiple data elements and perform the same operation on all elements simultaneously
HBM	High-Bandwidth Memory : expensive, specialized RAM designed to deliver very high bandwidth at the cost of higher latency
NVLink	NVIDIA's proprietary high-speed hardware interconnect for GPU-to-GPU and GPU-to-CPU communication
Infinity Fabric	AMD's high-speed (~100 Gb/s per lane) interconnect technology for inter-die and inter-node communication
Neuromorphic Computing	An experimental, qualitatively different approach to computation inspired by biological neural networks

0.3 Optimization: Basic Steps — Avoiding Trivial Inefficiencies

0.3.1 Lecture Context

This lecture belongs to the optimization series and sits between the introductory “First Things First” material and the deeper treatments of branches, pipelines, and loops. Its focus is on **trivial inefficiencies** — performance pitfalls that are easy to introduce and, once recognized, straightforward to eliminate. The lecture proceeds through a concrete running example, progressively refining a piece of code to illustrate six fundamental optimization principles.

The outline of the optimization sub-series is:

1. First things first — general concepts and compiler preliminaries
 2. **Avoid the avoidable inefficiencies** (this lecture)
 3. Branches
 4. Pipelines
 5. Loops
-

0.3.2 Running Example: Grid-Based Neighbor Search

To ground the discussion in a realistic scenario, the lecture introduces a **grid-based proximity search** problem:

1. A set of N_p random data points is distributed on a 2D (or 3D) plane, which is subdivided into a regular grid of $N_g \times N_g \times N_g$ cells.
2. For each point p , the algorithm must identify all grid cells whose center lies within a given radius R from p , and perform some operation for each such cell.

The naïve implementation uses four nested loops — one over points and three over grid dimensions:

```
for (p = 0; p < Np; p++)
  for (i = 0; i < Ng; i++)
    for (j = 0; j < Ng; j++)
      for (k = 0; k < Ng; k++) {
        dist = sqrt(
          pow(x[p] - (double)i/Ng - half_size, 2) +
          pow(y[p] - (double)j/Ng - half_size, 2) +
          pow(z[p] - (double)k/Ng - half_size, 2));
        if (dist < R)
          /* do something */;
      }
```

This code is correct but contains multiple layers of avoidable inefficiency. The lecture systematically removes them one by one.

0.3.3 Optimization Principle 1: Avoid Expensive Function Calls

0.3.3.1 The Cost of Mathematical Functions Certain library function calls carry a **disproportionately high computational cost** relative to the underlying mathematical operation. The most common offenders in numerical code are:

Function	Issue
<code>sqrt()</code>	Involves iterative approximation (e.g., Newton–Raphson); far more expensive than basic arithmetic
<code>pow(x, 2)</code>	A general-purpose exponentiation routine; vastly more expensive than a simple multiplication <code>x * x</code>
Floating-point division (<code>/</code>)	Typically 3–10× more expensive than multiplication on modern hardware

0.3.3.2 Step 1: Eliminate `sqrt()` The comparison `dist < R` is mathematically equivalent to $\text{dist}^2 < R^2$, since both sides are non-negative. Squaring both sides eliminates the need for `sqrt()` entirely:

```
dist2 = pow(x[p] - (double)i/Ng - half_size, 2) +
        pow(y[p] - (double)j/Ng - half_size, 2) +
        pow(z[p] - (double)k/Ng - half_size, 2);
if (dist2 < R2)
    /* do something */;
```

where `R2 = R * R` is precomputed once.

0.3.3.3 Step 2: Eliminate `pow()` The call `pow(expr, 2)` invokes the general-purpose power function, which internally handles arbitrary real exponents. For the special case of squaring, a simple multiplication is both **exact** and orders of magnitude faster:

```
dx = x[p] - (double)i/Ng - half_size;
dy = y[p] - (double)j/Ng - half_size;
dz = z[p] - (double)k/Ng - half_size;
dist2 = dx*dx + dy*dy + dz*dz;
```

0.3.3.4 Step 3: Replace Division with Multiplication The expression `(double)i / Ng` performs a floating-point division at every iteration. Since `Ng` is loop-invariant, the reciprocal $1/N_g$ can be precomputed once:

```
double Ng_inv = 1.0 / Ng;
// ...
dx = x[p] - (double)i * Ng_inv - half_size;
```

This replaces an expensive division with a cheap multiplication inside the innermost loop.

Key insight: the compiler can typically perform these substitutions for **integer** arithmetic, but it **cannot** do so for floating-point arithmetic due to the non-associativity of IEEE floating-point operations (discussed in detail below).

0.3.4 Optimization Principle 2: Hoisting Loop-Invariant Expressions

Hoisting (also called **loop-invariant code motion**) is the practice of moving computations that produce the same result on every iteration **outside** the loop in which they reside.

0.3.4.1 Identifying the Redundancy In the naïve loop, the expression `(double)i * Ng_inv + half_size` is recomputed at every iteration of the j and k loops, despite depending only on i . Similarly, the j -dependent expression is recomputed for every k . The total number of redundant evaluations is:

$$N_{\text{redundant}} = N_g^3 + N_g^2 + N_g$$

Each redundant evaluation involves a multiplication, an addition, and a memory access — all of which can be saved.

0.3.4.2 Hoisted Version By restructuring the loop nest and computing partial distances at the appropriate level:

```
for (i = 0; i < Ng; i++) {
    double dx2 = x[p] - (double)i * Ng_inv - half_size;
    dx2 = dx2 * dx2;

    for (j = 0; j < Ng; j++) {
        double dy2 = y[p] - (double)j * Ng_inv - half_size;
        dy2 = dy2 * dy2;
        double dist2_xy = dx2 + dy2;

        for (k = 0; k < Ng; k++) {
            double dz = z[p] - (double)k * Ng_inv - half_size;
            double dist2 = dist2_xy + dz * dz;
            if (dist2 < Rmax2)
                /* do something */;
        }
    }
}
```

The x -component distance is computed once per i -iteration, the y -component once per j -iteration, and only the z -component is computed in the innermost loop.

0.3.4.3 Further Improvement: Precomputed Grid Coordinates An additional optimization precomputes all grid coordinate values into an array:

```
double ijk[Ng];
for (i = 0; i < Ng; i++)
    ijk[i] = i * Ng_inv + half_size;
```

This replaces all arithmetic inside the loop with a single **array lookup**, which is both faster and clearer.

0.3.5 Optimization Principle 3: Clarify Variable Scope

Declaring variables at the **narrowest possible scope** — that is, as close as possible to their point of use — serves two complementary purposes.

0.3.5.1 Benefit for the Compiler All local variables reside on the **stack**. The compiler must decide where each variable should be placed within the stack frame so that it is “hot” (present in cache) at the right moment. When variables are declared with minimal scope, the compiler gains more information about **liveness ranges** (the interval during which a variable holds a meaningful value), enabling better decisions about:

- **Stack layout:** variables used together can be placed adjacently in memory, improving **spatial locality**
- **Register allocation:** variables with non-overlapping lifetimes can share the same register

```
for (int i = 0; i < Ng; i++) {
    double dx2 = x[p] - (double)i * Ng_inv - half_size;
    dx2 *= dx2;
    for (int j = 0; j < Ng; j++) {
        double dy2 = y[p] - (double)j * Ng_inv - half_size;
        double dist2_xy = dx2 + dy2 * dy2;
        for (int k = 0; k < Ng; k++) {
            double dz = z[p] - (double)k * Ng_inv - half_size;
            double dist2 = dist2_xy + dz * dz;
            if (dist2 < Rmax2)
                /* do something */;
        }
    }
}
```

0.3.5.2 Benefit for the Programmer Tight scoping forces the programmer to **think clearly** about what each variable represents and when it is needed. This has practical consequences:

- It prevents accidental **dependency chains** — for example, a variable generically named **temp**, reused across unrelated computations, may carry stale values into contexts where it was not intended
- It makes the code easier to **maintain, debug, and reason about**
- It reduces the risk of subtle bugs that arise from variable reuse

“Being a good programmer is all about thinking clearly and sharply.”

0.3.6 Optimization Principle 4: Suggest Register Placement

The **register** storage class hint tells the compiler that a particular variable is accessed frequently and would benefit from being held in a CPU register rather than on the stack:

```
double register Ng_inv = 1.0 / Ng;
// ...
double register dist2_xy = dx2 + dy2;
// ...
double register dz = z[p] - (double)k * Ng_inv - half_size;
double register dist2 = dist2_xy + dz * dz;
```

Important caveat: this is a **suggestion**, not a directive. The compiler performs its own register allocation analysis and may override the programmer's hint if it determines a different allocation is superior. Modern optimizing compilers are generally very good at register allocation, so explicit **register** hints are rarely necessary — but they can occasionally help, and they serve as useful **documentation** of the programmer's intent.

0.3.7 Compiler Limitations and the Compiler's Perspective

While modern compilers can handle many of the optimizations described above automatically (hoisting, scope narrowing, some strength reductions), there is one critical exception:

The compiler cannot rearrange floating-point operations, even when a mathematically equivalent rearrangement would be more efficient.

This limitation is rooted in the properties of **IEEE 754 floating-point arithmetic**.

0.3.8 Floating-Point Arithmetic: Non-Associativity

0.3.8.1 The Fundamental Issue On a digital system with a **finite number of digits**, floating-point arithmetic does not obey the same algebraic rules as real-number arithmetic:

Property	Integer arithmetic (2's complement)	Floating-point arithmetic
Commutativity of + and ×	Yes	Yes (between two operands)
Associativity of + and ×	Yes	No

The critical consequence is:

$$(a + b) + c \neq a + (b + c)$$

in general, for IEEE floating-point numbers a , b , c .

0.3.8.2 Why Associativity Fails: A Concrete Example Consider a floating-point system with **3 digits of mantissa precision** in base 10. The representable values have the form $d_1.d_2d_3 \times 10^e$.

- $1.00 + 0.01 = 1.01$ — this is exact and fits within 3 digits
- $1.00 + 0.001 = 1.00$ — the true result 1.001 requires 4 digits and is **rounded** to 1.00

This means that **the order in which additions are performed changes the result**:

Left-to-right accumulation:

$$\underbrace{0.001 + 0.001 + \cdots + 0.001}_{10 \text{ terms}} = 0.01$$

Adding these small terms together first preserves their contribution, yielding 0.01. Then:

$$0.01 + 1.00 = 1.01$$

Right-to-left accumulation:

$$1.00 + 0.001 = 1.00, \quad 1.00 + 0.001 = 1.00, \quad \dots$$

Each small term is absorbed into the larger number, and the final result is 1.00.

0.3.8.3 Implications for the Compiler Because reordering floating-point operations can **change numerical results**, the compiler is **forbidden** from performing optimizations that alter the evaluation order of floating-point expressions — even when the rearranged form would be mathematically equivalent and computationally faster.

This restriction applies to:

- Reordering additions or multiplications
- Factoring out common subexpressions that involve FP arithmetic
- Replacing a/b with $a * (1/b)$ across different contexts
- Vectorizing reductions that require associative regrouping

0.3.8.4 Relaxing IEEE Compliance: Fast-Math Flags Every major compiler provides flags to **relax IEEE floating-point rules**, thereby enabling aggressive FP optimizations:

Compiler	Flag
GCC	<code>-ffast-math</code> (umbrella), <code>-fassociative-math</code> , <code>-ffinite-math-only</code> , etc.
Clang	<code>-ffast-math</code> , <code>-funsafe-math-optimizations</code>
ICC	<code>-fp-model fast</code>

These flags allow the compiler to:

- Assume associativity of FP operations
- Ignore special values (NaN, Inf, denormals, signed zeros)
- Reorder and vectorize FP operations freely

Warning: enabling fast-math also disables safety checks such as `isnan(x)` and `isinf(x)`. Moreover, once operations are reordered, the numerical results may differ from the IEEE-compliant version — and tests performed *without* fast-math flags are not necessarily representative of behavior *with* them, since the reordering may introduce new edge cases.

The programmer must be confident that the code does not encounter special FP values before enabling these flags.

0.3.9 Optimization Principle 5: Avoid Unnecessary Repeated Checks

0.3.9.1 The `strlen()` Anti-Pattern Consider this function that searches for a character in a string:

```
char *find_char_in_string(char *string, char c) {
    int i = 0;
    while (i < strlen(string))
        if (string[i] == c)
            break;
        else
            i++;
    if (i < strlen(string))
        return &string[i];
    else
        return NULL;
}
```

The critical performance flaw is the **repeated call to `strlen()`** inside the loop condition. The function `strlen()` itself traverses the entire string to find the null terminator — it has $O(n)$ complexity. Since it is called at **every iteration**, the overall complexity of this loop degrades to $O(n^2)$.

The compiler **cannot** optimize this away because:

- It cannot prove that the string content does not change between iterations (the memory pointed to by `string` could be modified externally, e.g., by another thread or a signal handler)
- `strlen()` is an opaque function call from the compiler's perspective in many contexts

0.3.9.2 The Fix: Cache the Invariant A simple refactoring stores the string length in a local variable:


```

char *find_char_in_string(char *string, char c) {
    int i = 0;
    int len = strlen(string);
    while (i < len)
        if (string[i] == c)
            break;
        else
            i++;
    if (i < len)
        return &string[i];
    else
        return NULL;
}

```

This reduces the complexity back to $O(n)$.

0.3.9.3 An Even Better Version: Pointer-Based Traversal A further refinement avoids indexing altogether and uses direct pointer arithmetic:

```

char *find_char_in_string(char *string, char c) {
    char *pos = string;
    while ((*pos != '\0') && (*pos != c))
        pos++;
    return (*pos == '\0' ? NULL : pos);
}

```

This version is more efficient for several reasons:

- **No separate length computation:** the null terminator check is integrated into the traversal
- **No integer index:** eliminates the overhead of maintaining and incrementing a counter variable, and avoids the array indexing operation `string[i]` (which involves an addition and a dereference)
- **Single pass:** the string is traversed exactly once, with a single dereference per iteration
- **Simpler control flow:** fewer branches and comparisons per iteration

0.3.10 Optimization Principle 6: Avoid Unnecessary Memory References

0.3.10.1 The Problem: Pointer Dereferencing in a Loop Consider a simple vector reduction that accumulates values through a pointer:

```

void reduce_vector(int n, double *array, double *sum) {
    for (int i = 0; i < n; i++)
        *sum += array[i];
    return;
}

```

At each iteration, the code:

1. **Reads** the current value of `*sum` from memory
2. **Adds** `array[i]` to it
3. **Writes** the result back to the memory location pointed to by `sum`

The generated assembly (at -O1) confirms this triple memory access per iteration:

```
movsd    xmm0, [sum]           ; load *sum
addsd    xmm0, [array+rax]     ; add array[i]
movsd    [sum], xmm0          ; store back to *sum
add      rax, 8                ; advance array pointer
cmp      rax, n                ; check loop bound
```

The **load and store of `*sum`** are repeated every iteration because the compiler cannot assume that `sum` and `array` do not alias — the memory region pointed to by `sum` might overlap with `array`, so the written-back value must be visible in memory before the next read.

0.3.10.2 The Fix: Use a Local Accumulator Introducing a **local temporary variable** allows the compiler to keep the accumulator in a register:

```
void reduce_vector(int n, double *array, double *sum) {
    double temp = 0;
    for (int i = 0; i < n; i++)
        temp += array[i];
    *sum = temp;
    return;
}
```

The generated assembly is now tighter:

```
addsd    xmm0, [array+rax]     ; add array[i] to register
add      rax, 8                ; advance pointer
cmp      rax, rdx              ; check loop bound
```

The accumulator `temp` resides in a **register** (`xmm0`) throughout the loop, eliminating the redundant load–store pair. The final result is written to `*sum` only once, after the loop completes. This saves **two memory operations per iteration** — a substantial improvement for large arrays.

This optimization connects directly to the **memory aliasing** discussion: the compiler’s inability to prove non-aliasing between `sum` and `array` forces conservative behavior. By introducing a local variable that provably does not alias any pointer, the programmer removes this ambiguity.

0.3.11 Summary of Basic Optimization Principles

#	Principle	Key Action
1	Avoid expensive function calls	Replace <code>sqrt()</code> , <code>pow()</code> , and division with algebraically equivalent cheaper operations
2	Hoist loop-invariant expressions	Move computations that do not change across iterations outside the loop
3	Clarify variable scope	Declare variables at the narrowest scope to aid both the compiler and the programmer
4	Suggest register placement	Use <code>register</code> hints for frequently accessed variables
5	Avoid unnecessary repeated checks	Cache invariant values; do not recompute what does not change
6	Avoid unnecessary memory references	Use local accumulators instead of repeated pointer dereferences

A cross-cutting theme pervades all six principles: **the compiler is remarkably capable, but it is not omniscient**. It can often perform hoisting, scope analysis, and register allocation automatically — but it **cannot** rearrange floating-point operations, cannot prove that opaque function calls are idempotent, and cannot resolve pointer aliasing in all cases. The programmer’s role is to write code that is **clear enough** for the compiler to optimize aggressively, and to manually intervene where the compiler’s hands are tied.

0.4 The Execution Model: Heap and Stack

0.4.1 Lecture Context

This lecture addresses a fundamental question: **what happens when you execute a program on a machine** — whether a personal laptop or a large Tier-0 HPC facility? Before any optimization can take place, one must understand how compiled code interacts with the operating system, how it gains access to memory and CPU resources, and how the two primary memory regions — the **stack** and the **heap** — govern variable storage and data access.

The lecture focuses on ***nix systems** (Linux, Unix), which are the *de facto* standard in HPC environments. However, the conceptual framework generalizes to other operating systems at the level of abstraction discussed here.

0.4.2 The Virtual Memory Model

0.4.2.1 The Memory Sandbox When a program is executed, the operating system provides a **virtual address space** — a “memory sandbox” — that appears homogeneous and contiguous to the running program. This abstraction hides the hardware-specific details of physical memory, enhancing both programming ease and portability.

The size of the addressable virtual space depends on the architecture and the operating system:

Architecture	Bits Used for Addressing	Addressable Space
32-bit	32	4 GB
64-bit (48-bit used)	48	256 TB
64-bit (56-bit used)	56	65,536 TB
64-bit (full)	64	16 EB (exabytes)

At the moment a process is created, only a fraction of this space is actually backed by physical memory. The addressable range represents the **maximum possible extent**, not the immediately available allocation.

0.4.2.2 Virtual-to-Physical Address Translation The virtual addresses used by the program must be translated into **physical addresses** that the memory controller can use to access the actual DRAM. This translation is performed through the **paging mechanism**:

- Physical memory is divided into fixed-size **pages**. Typical page sizes are:
 - **4 KB** for systems with modest total RAM
 - **2 MB** or **1 GB (huge pages)** for systems with very large RAM, commonly found in HPC
- Each physical page can be mapped to **multiple virtual pages** — for instance, a shared library loaded by several processes will appear in each process’s virtual address space, yet occupy only one region of physical memory
- The mappings are described in a hierarchical **page table**, maintained by the kernel, containing a **Page Table Entry (PTE)** for each mapped page

0.4.2.3 The Translation Lookaside Buffer (TLB) Because page table lookups can be expensive (requiring multiple memory accesses to traverse the hierarchical table), a specialized cache called the **Translation Lookaside Buffer (TLB)** accelerates the translation:

Property	Typical Value
Entries	Thousands (typically ~12 bits of addressing)
Hit time	~1 cycle
Miss penalty	~100 cycles
Typical miss rate	~1–2%

The TLB lookup cycle proceeds as follows:

1. The CPU issues a virtual address
2. The TLB is checked:
 - **TLB hit**: the physical address is returned immediately (~1 cycle)
 - **TLB miss**: the page table must be walked, the translation is computed (~100 cycles), and the result is registered in the TLB for future use

Despite the low miss rate, TLB misses carry a **high per-event cost**, making them a non-negligible factor in performance-sensitive code — particularly when data access patterns are irregular or span very large memory regions.

0.4.3 The Standard Execution Model

0.4.3.1 Memory Layout of a Process When a compiled program is loaded and executed, the operating system organizes its virtual address space into several well-defined **segments**. In a simplified but effective representation (using the classic 32-bit layout for illustration, though 64-bit systems extend the same model to a vastly larger address range):

Segment	Address Range (32-bit)	Contents
Kernel space	0xC0000000 – 0xFFFFFFFF	Reserved for the OS kernel; inaccessible to user code
Stack	Below kernel space, growing downward	Local variables, function arguments, return addresses
Heap	Above BSS, growing upward	Dynamically allocated data (<code>malloc</code> , <code>calloc</code>)
BSS (Block Started by Symbol)	Above initialized data	Uninitialized global variables
Initialized data	Above text	Initialized global variables (e.g., <code>double pi = 3.14159...</code>)
Text (code)	Starting at ~0x08048000	The compiled machine code of the program

The **stack grows downward** (toward lower addresses) while the **heap grows upward** (toward higher addresses). They expand toward each other in the virtual address space.

0.4.3.2 The Text Segment The **text segment** contains the actual machine code — the compiled instructions of all the functions in the program. When the CPU executes the program, it fetches instructions from this region. All routines (`main()`, helper functions, library calls) reside here.

0.4.3.3 The Initialized Data Segment This segment holds **global variables that have been assigned an initial value** at compile time. For example:

```
double pi_greek = 3.14159265358979323846;
```

This variable’s value is known at compile time and is stored directly in the binary, loaded into this segment at program startup.

0.4.3.4 The BSS Segment The **BSS segment** holds **global variables that are declared but not initialized**:

```
double result_of_calculation;  
int iteration_counter;
```

These variables have no meaningful value at compile time. The operating system typically zero-initializes this segment when the process starts.

0.4.4 Stack and Heap: Fundamental Characteristics

0.4.4.1 The Stack The **stack** is a region of local memory designed to hold **local variables** for each function invocation. Its key characteristics are:

- **Contents:** function arguments, return addresses, local variables (counters, pointers, temporary values)
- **Scope:** strictly limited — only the current function (and its callees, through pointer passing) can access a given stack frame
- **Management:** automatically managed by the CPU using two dedicated registers:
 - **BP** (Base Pointer, also called Frame Pointer): points to the beginning of the current function’s local variable area
 - **SP** (Stack Pointer): points to the current “top” of the stack (the lowest occupied address, since the stack grows downward)
- **Natural use:** **static allocation** of local variables whose size is known at compile time

0.4.4.2 The Heap The **heap** is a large, flexible memory region designed to host the bulk of a program’s data. Its key characteristics are:

- **Contents:** global variables, large data structures, any data whose size is determined at runtime
- **Scope:** global — data on the heap can be accessed from any function, provided a valid pointer exists
- **Management:** explicitly managed by the programmer through allocation (`malloc`, `calloc`) and deallocation (`free`) calls

- **Natural use: dynamic allocation** of data whose size or lifetime cannot be determined at compile time

0.4.5 Stack Mechanics in Detail

0.4.5.1 The Stack Frame Each function invocation creates a **stack frame** (also called an **activation record**) containing:

1. **Arguments** of the function (with a caveat: on x86_64, the first 6 arguments that fit in 64 bits — numerical values and pointers — are passed via **registers**, not the stack; only the 7th argument onward, or arguments exceeding 64 bits, are placed on the stack)
2. **Return address**: the instruction address in the caller to which control should return
3. **Local variables**: all variables declared within the function body

The **BP register** points to the boundary between the return address and the local variables, while the **SP register** points to the current top (lowest address) of the stack.

0.4.5.2 Function Call Sequence Consider a simple scenario where `func1()` calls `func2()`:

```
func1() {
    // ...
    func2();
    // ...
}
```

Before the call — while `func1()` is executing:

Stack Contents (top to bottom, i.e., low to high addresses)

Local variables of `func1()` ← SP points here
 Return address (to `func1()`'s caller) ← BP points here
 Arguments of `func1()`

After the call — when `func2()` is executing:

Stack Contents

Local variables of `func2()` ← SP points here
 Return address (to `func1()`) ← BP points here
 Arguments of `func2()`
 Local variables of `func1()`
 Return address (to `func1()`'s caller)
 Arguments of `func1()`

The stack has grown downward to accommodate `func2()`'s frame. When `func2()` returns, SP and BP are restored to their previous values, effectively **destroying** `func2()`'s frame.

0.4.5.3 Addressing Variables on the Stack Variables on the stack are accessed via **fixed offsets** from either BP or SP:

$$\text{address} = \text{BP} - \text{offset} - \text{sizeof}(\text{variable})$$

or equivalently:

$$\text{address} = \text{SP} + \text{offset}$$

where **offset** accounts for all other data between the register and the target variable. For instance, in a function with local variables `double a, int j, int k` (listed from BP downward):

- The offset of `k` from SP is 0
- The offset of `j` from SP is `sizeof(k) = 4` bytes
- The offset of `a` from BP is `sizeof(a) + sizeof(j) = 12` bytes

This offset-based addressing is efficient because BP is already resident in a CPU register — no pointer dereference to an arbitrary heap location is needed.

0.4.5.4 Variable Byte Order and Memory Layout A variable occupies a contiguous sequence of bytes starting at its **lowest memory address** and extending toward higher addresses. The internal byte ordering within a multi-byte variable follows the system's **endianness** (little-endian on x86_64, meaning the least significant byte occupies the lowest address).

0.4.6 Stack Scope and Lifetime

0.4.6.1 Scope Isolation Between Functions The stack's scope rules have important practical consequences, best illustrated by a concrete call sequence:

```
function_1() { /* ... */ }
function_3() { /* ... */ }
function_2() { /* ... */ function_3(); /* ... */ }
int main() { function_1(); function_2(); /* ... */ return 0; }
```

The execution proceeds as follows:

1. `main()` calls `function_1()`: the stack grows to hold `function_1()`'s frame
2. `function_1()` returns: its stack frame is **destroyed** — its local data no longer exist
3. `main()` calls `function_2()`: the stack grows again, **reusing the same memory** previously occupied by `function_1()`'s frame. Consequently, `function_2()` **cannot access** `function_1()`'s data — they have been overwritten

4. `function_2()` calls `function_3()`: the stack grows further. `function_3()` **can** access `function_2()`'s stack (e.g., if `function_2()` passes a pointer to a local variable as an argument)

Key insight: callee functions can access the stack of their callers (through passed pointers), but sibling or previously returned functions' data are irrecoverable.

0.4.6.2 Why the Stack Should Remain Small The stack is intentionally kept small (typical default size: 8 MB, queryable via `ulimit -s`). This design serves multiple purposes:

- **Cache friendliness:** a small stack means local variables are likely to reside in cache, yielding fast access
- **Deep call support:** the limited size is sized to accommodate deep call chains (e.g., recursive functions) rather than large data storage
- **Design signal:** if you find yourself needing to enlarge the stack, it is likely a sign that the program's design should be reconsidered — large data belongs on the heap

0.4.7 Dynamic Allocation on the Stack: `alloca()`

In situations where a **locally scoped array** of runtime-determined size is needed — and the size is expected to be small — the `alloca()` function provides dynamic allocation **on the stack** rather than the heap:

```
int some_function(int n, double *data) {
    // n is expected to be small
    double *local_temp = (double *)alloca(n * sizeof(double));

    // ... operations using local_temp ...

    // No free() needed - memory is automatically reclaimed
    // when the function returns
    return result;
}
```

The effect is that SP is moved further downward to accommodate the requested bytes. When the function returns, the entire frame (including the `alloca()`-allocated region) is automatically reclaimed.

Important: calling `free()` on an `alloca()`-allocated pointer would raise an exception, since the memory was not obtained from the heap allocator.

0.4.8 Stack Size Management

0.4.8.1 Querying and Modifying Stack Limits The operating system imposes both a **soft limit** (user-adjustable) and a **hard limit** (system-enforced maximum) on stack size:

Command	Purpose
<code>ulimit -s</code>	Display the current soft limit (in KB)
<code>ulimit -s <value></code>	Set a new soft limit
<code>ulimit -Hs</code>	Display the hard limit (may be <code>unlimited</code> or <code>-1</code>)

0.4.8.2 Programmatic Stack Management Stack limits can also be queried and modified from within C code:

```
#include <sys/resource.h>

struct rlimit stack_limits;
getrlimit(RLIMIT_STACK, &stack_limits);

// Set a new limit (if permitted by the hard limit)
struct rlimit new_limits = stack_limits;
new_limits.rlim_cur = new_limit_value;
setrlimit(RLIMIT_STACK, &new_limits);
```

The hard limit value of `-1` conventionally means “unlimited” — this works because stack sizes are unsigned quantities, so `-1` in two’s complement represents the maximum representable unsigned value.

0.4.9 Stack vs. Heap: Performance Comparison

0.4.9.1 The Fundamental Difference The core performance distinction between stack and heap access lies in **how addresses are computed**:

- **Stack access:** variables are addressed via **static offsets** from the BP or SP registers, which are always resident in CPU registers. The address computation is a simple addition of a constant offset — no pointer dereference is needed.
- **Heap access:** variables are addressed via **pointers** stored in memory. Accessing a heap-allocated array element typically requires loading the base pointer from memory, computing the element offset, and then dereferencing the result.

0.4.9.2 Assembly-Level Analysis Consider three ways to zero-initialize an array, and their corresponding assembly output (compiled without optimization):

1. Array on the stack:

```
float on_stack[N];
for (int i = 0; i < N; i++)
    on_stack[i] = 0;
```

Assembly:

```
MOV  eax, DWORD PTR -024[rbp]      ; load loop counter i
CDQE
MOVSS DWORD PTR -1024[rbp+rax*4], 0 ; store 0 to stack array element
```

The array element is accessed directly via a fixed offset from `rbp` — a single indexed memory operation.

2. Array on the heap (index notation):

```
float *on_heap = (float *)calloc(N, sizeof(float));
for (int i = 0; i < N; i++)
    on_heap[i] = 0;
```

Assembly:

```
MOV  eax, DWORD PTR -148[rbp]      ; load counter i
CDQE
LEA  rdx, 0[0+rax*4]                ; compute byte offset of i-th element
MOV  rax, QWORD PTR -36[rbp]        ; load base pointer of array from stack
ADD  rax, rdx                       ; compute final address
MOVSS DWORD PTR [rax], xmm0         ; store 0 to array element
```

This requires **more instructions**: the base address of the array must be loaded from the stack, the offset computed, and the two added together before the store can occur.

3. Array on the heap (pointer arithmetic):

```
float *on_heap = (float *)calloc(N, sizeof(float));
register float *ptr = on_heap - 1;
register float *stop = on_heap + N;
for (; ++ptr < stop; )
    *ptr = 0;
```

Assembly:

```
MOVSS DWORD PTR [rbx], xmm0        ; store 0 directly via pointer in register
ADD  rbx, 4                        ; advance pointer by sizeof(float)
```

This is the most efficient heap access pattern — the pointer resides in a register (`rbx`), and each iteration requires only a store and a pointer increment. **No index computation or base-address loading** is needed.

0.4.9.3 Comparative Summary

Access Method	Instructions per Iteration	Key Overhead
Stack (indexed)	3–4	Counter load + indexed addressing
Heap (indexed)	5–6	Base pointer load + offset calculation + addition
Heap (pointer arithmetic)	2	Minimal — pointer in register

The comparison reveals a critical insight: **the heap can be as fast as (or faster than) the stack**, depending on how the access is structured. The decisive factor is not *where* the data resides, but *how* the code accesses it. With pointer-based traversal and **register** hints, heap access can be reduced to a minimal instruction footprint.

Furthermore, when compiler optimizations are enabled (`-O2` or `-O3`), the compiler typically transforms even the indexed stack access into code resembling the hand-optimized pointer-based heap version, largely **erasing the performance difference** between the three approaches.

0.4.10 Generating and Inspecting Assembly Output

Understanding the assembly code generated by the compiler is an essential skill for HPC optimization. GCC provides several ways to produce assembly listings:

Command	Output
<code>gcc foo.c -S</code>	Basic assembly listing (<code>foo.s</code>)
<code>gcc foo.c -S -fverbose-asm</code>	Assembly with source-level annotations
<code>gcc foo.c -fverbose-asm -Wa,-adhln=foo.s</code>	Detailed listing with high-level source interleaved

The `-Wa` flag passes options directly to the assembler. Useful sub-options include:

Sub-option	Meaning
<code>a</code>	Turn on listings
<code>d</code>	Omit debugging directives
<code>h</code>	Include high-level source code
<code>l</code>	Include assembly code
<code>n</code>	Omit forms processing
<code>s</code>	Include symbols
<code>=FILE</code>	Redirect listing to FILE

Inspecting the generated assembly — particularly with and without optimization flags — provides direct insight into which optimizations the compiler applies and where manual intervention may be beneficial.

0.4.11 Practical Considerations and Design Implications

0.4.11.1 When to Use the Stack

- Local variables with **known size at compile time** and **limited scope**
- Small temporary buffers needed only within a single function
- Loop counters, accumulators, and intermediate results

0.4.11.2 When to Use the Heap

- Data whose **size is determined at runtime** (e.g., input-dependent arrays)
- Large data structures that would exceed the stack size limit
- Data that must **persist beyond the lifetime** of the function that creates it
- Global or shared data structures accessible from multiple functions

0.4.11.3 The Danger of Returning Stack Addresses A common and dangerous error is returning a pointer to a local (stack-allocated) variable from a function:

```
int *function_1(void) {  
    int local_var = 42;  
    return &local_var; // WARNING: address of local variable  
}
```

The compiler will emit a warning: “*function returns address of local variable.*” Once `function_1()` returns, its stack frame is destroyed and the memory at `&local_var` will be overwritten by subsequent function calls. Dereferencing the returned pointer results in **undefined behavior**.

0.4.11.4 Stack Smashing **Stack smashing** occurs when a program writes beyond the bounds of a stack-allocated buffer, corrupting adjacent stack data (return addresses, saved registers, other local variables). This is both a correctness bug and a well-known **security vulnerability** (exploited in classic buffer overflow attacks). Modern compilers include **stack protector** mechanisms (`-fstack-protector`) that insert canary values to detect such overflows at runtime.

0.5 Cache and Memory: Detailed Mechanisms

0.5.1 Cache–DRAM Mapping Strategies

The fundamental mapping problem — how to place the contents of a $\sim 10^{10}$ -byte main memory into a $\sim 10^4$ -byte cache — admits three canonical solutions. In all cases, both RAM and cache are subdivided into fixed-size **blocks** (typically 64 bytes each), called **cache lines**. Data is never loaded byte-by-byte; an entire line is fetched on every cache fill.

0.5.1.1 Direct Mapping In a **direct-mapped** cache, each memory block can reside in exactly one predetermined cache line:

$$\text{cache_line} = \text{block_number} \bmod \text{number_of_cache_lines}$$

Advantage	Disadvantage
Extremely efficient placement and lookup — only one location to check	Maximizes conflict misses : many memory addresses compete for the same single cache line

0.5.1.2 Fully Associative Mapping In a **fully associative** cache, any memory block can be placed in **any** cache line.

Advantage	Disadvantage
Minimizes conflict misses — any free line is eligible	Extremely inefficient lookup — every line must be checked to determine whether a given address is cached

0.5.1.3 N-Way Set-Associative Mapping The **n-way set-associative** cache is the dominant design in modern processors, combining the strengths of both extremes. The cache is divided into **sets**, each containing w lines (where w is the **associativity**, typically 4–8). A memory block maps to a unique set (as in direct mapping), but within that set it can occupy any of the w lines (as in full associativity):

$$\text{set} = \text{block_number} \bmod \frac{C}{L \times w}$$

where C is the total cache size, L is the line size, and w is the associativity.

Advantage	Disadvantage
Good trade-off: limited conflict misses due to w slots per set; efficient lookup since only w lines need checking	More complex hardware logic than direct mapping

A typical modern L1 data cache (32 KB, 64-byte lines, 8-way associative) has:

$$\text{number of sets} = \frac{32,768}{64 \times 8} = 64 \text{ sets}$$

0.5.1.4 Address Decomposition For a cache with line size $L = 2^b$ bytes, 2^s sets, and w -way associativity, a memory address is decomposed into three fields:

$$\left[\underbrace{\text{tag}}_{\text{remaining high bits}} \mid \underbrace{\text{set index}}_{s \text{ bits}} \mid \underbrace{\text{block offset}}_{b \text{ bits}} \right]$$

- The **block offset** (b least significant bits) identifies the byte position within a cache line
- The **set index** (s bits) determines which set the address maps to — this is essentially a direct-mapping step
- The **tag** (remaining upper bits) is stored alongside the cache line to enable identification of which memory block currently occupies each line

For a typical configuration ($L = 64$ B, $b = 6$; 128 sets, $s = 7$), the first 13 bits of the address determine the byte-in-line and set, while the tag records the full provenance of the cached data.

0.5.2 Memory Access Patterns and Cache Behavior

The performance of a program depends critically on how its memory access pattern interacts with the cache. Three carefully constructed examples illustrate this.

0.5.2.1 Example Setup Consider a simplified direct-mapped cache with 16 bytes total, two cache lines of 8 bytes each (holding two `float` values per line), and a cache-aligned array `float X[8]`.

0.5.2.2 Pattern 1: Sequential Access

```
for (int j = 0; j < 2; j++)
    for (int i = 0; i < 8; i++)
        access(X[i]);
```

The hit-miss pattern is: **M H M H M H M H M H M H M H M H** — every other access is a miss (all misses are **compulsory**, occurring on first contact with each cache line). The miss rate is **50%**.

0.5.2.3 Pattern 2: Strided Access (Worst Case)

```
for (int j = 0; j < 2; j++) {
    for (int i = 0; i < 7; i += 2)
        access(X[i]);           // even indices
    for (int i = 1; i < 8; i += 2)
        access(X[i]);           // odd indices
}
```

The hit-miss pattern is: **M M M M M M M M M M M M M M M M** — every single access is a miss. The miss rate is **100%**. This occurs because accessing all even-indexed elements evicts the lines holding odd-indexed elements, and vice versa — a classic case of **conflict misses** caused by strided access that defeats spatial locality.

0.5.2.4 Pattern 3: Block-Oriented Access (Best Case)

```
for (int k = 0; k < 2; k++)
    for (int i = 0; i < 2; i++)
        for (int j = 4*k; j < (k+1)*4; j++)
            access(X[j]);
```

The hit-miss pattern is: **M H M H H H H H M H M H H H H H** — a miss rate of only **25%**. By accessing the array in contiguous blocks that fit within a cache line, and revisiting those blocks before they are evicted, this pattern maximizes both temporal and spatial locality.

Key insight: for the same total number of accesses to the same data, miss rates can range from 25% to 100% depending solely on the access pattern.

0.5.3 The Memory Mountain

The **memory mountain** is a three-dimensional performance landscape that visualizes how achieved memory throughput (effective bandwidth) varies as a function of two parameters:

1. **Working set size** — the total amount of data accessed (plotted on a logarithmic scale, in KB)
2. **Stride** — the distance between consecutively accessed elements (from 1, meaning contiguous access, upward)

The measurement procedure traverses an array of a given size with a given stride, measuring the effective bandwidth achieved.

Topography:

- The **peak** (highest bandwidth) occurs for small working sets with unit stride — the data fits entirely in L1 cache and full spatial locality is exploited
- **Ridges** at intermediate working set sizes correspond to the L2 and L3 cache boundaries
- The **deep valley** at large working set sizes and large strides corresponds to DRAM access with no locality — the worst-case performance regime

The memory mountain is a diagnostic fingerprint of a specific machine's memory hierarchy.

0.5.4 The Cache-Miss Signature

A complementary diagnostic technique is the **cache-miss signature**: measuring the cache miss rate as a function of array size for a simple sequential loop:


```
for (j = 0; j < size; j++)
    array[j] = 2.3 * array[j] + 1.2;
```

Three distinct regimes emerge:

Regime	Condition	Behavior
L1 resident	Array fits in L1	Essentially zero misses after warm-up; all data are served from L1
L2 resident	Array exceeds L1, fits in L2	Compulsory misses at the rate of $1/(\text{elements per line})$; e.g., for 64-byte lines holding 8 doubles, the miss rate is $1/8 = 12.5\%$
DRAM bound	Array exceeds all cache levels	Every access to a new cache line is a miss; performance degrades to DRAM bandwidth

This technique can be used to experimentally determine cache sizes on an unknown machine.

0.5.5 Achievable Bandwidth: A Rough Guide

Access Pattern	Notes	Effective Per-Core Bandwidth
Sequential, L1	Best case	10–15 GB/s
Sequential, L3	Data exceeds L1 but fits L3	5–10 GB/s
Sequential, DRAM	Data exceeds all caches	2–5 GB/s
Strided	Stride = cache line size	Sequential bandwidth $\div$ elements per line
Random (pointer chasing)	Latency-bound	~ 0.5 GB/s

0.5.6 Matrix Storage and Strided Access

0.5.6.1 Row-Major Order In C/C++, a 2D matrix is stored in **row-major order**: rows are laid out contiguously in memory. Traversing a matrix row by row corresponds to sequential memory access; traversing column by column corresponds to strided access with a stride of N elements ($N \times S$ bytes, where S is the element size).

For a 3D matrix `double M[n][m][l]`, the **innermost index** (k) varies fastest and traces memory contiguity: `M[i][j][k]` is contiguous with `M[i][j][k+1]`. The element `M[i][j][k]` is located at offset $i \times (m \times l) + j \times l + k$ from the base address.

Fortran uses the opposite convention (**column-major order**), where columns are contiguous in memory.

0.5.6.2 Matrix Transpose and Write Policies The naïve matrix transpose reads the source matrix in row-major order and writes the destination in column-major order (or vice versa):

```
for (int row = 0; row < N; row++)
    for (int col = 0; col < N; col++)
        B[col * N + row] = A[row * N + col];
```

One of the two matrix traversals is necessarily strided. The performance depends on whether the stride falls on the **read** or the **write** side, due to the different cache policies for writes.

0.5.6.3 Write-Through vs. Write-Back When a cached value is modified, the change must eventually propagate to main memory. Two policies exist:

Policy	Behavior	Implication
Write-through	Every write is immediately propagated to lower memory levels	Memory and cache are always consistent; generates high memory traffic
Write-back	The new value is stored only in the cache; it is written to memory only when the cache line is evicted	Reduces memory traffic but requires tracking modified (“dirty”) lines

Modern caches typically use **write-back** with **write-allocate**: when a write misses, the entire cache line is first loaded from memory (write-allocate), then modified in cache, and flagged as **dirty**. The dirty line is flushed to memory only upon eviction.

0.5.6.4 Why Strided Writes Are More Expensive With write-back + write-allocate, a strided write pattern is particularly costly:

1. Each write to a new column element accesses a **different cache line**, which must be loaded via write-allocate
2. The loaded line is flagged dirty after a single element is written
3. When the line is evicted (to make room for the next column’s line), the entire dirty line must be **written back** to memory
4. When the algorithm revisits the same line (next column pass), the line must be reloaded — the previous writeback was wasteful

In contrast, strided reads with contiguous writes allow each cache line to be fully filled before being written back in a single burst.

Practical consequence: for matrix transpose, $A[\text{row} * N + \text{col}] = B[\text{col} * N + \text{row}]$ (contiguous write, strided read) is generally faster than $B[\text{col} * N + \text{row}] = A[\text{row} * N + \text{col}]$ (strided write, contiguous read).

0.5.6.5 Three Performance Regimes for Matrix Transpose Let C be the cache size, L the cache line size, and N the matrix dimension:

Regime	Condition	Behavior
1	$2N^2 \times \text{data_size} < C$	Both matrices fit in cache; traversal order is irrelevant
2	$N \times L < C$	A fraction of the strided dimension fits in cache; moderate miss rate
3	$N > C/L$	Each strided access causes a miss and a write-allocate; severe performance drop — only one element per cache line is used

0.5.7 Cache-Associativity Conflicts (Cache Resonance)

At certain matrix sizes — specifically, when N is a power of 2 — anomalous performance drops occur due to **cache-associativity conflicts**.

0.5.7.1 The Mechanism For an n -way set-associative cache, accessing element $(i, j + 1)$ of a column-traversed matrix involves a stride of $N \times W$ bytes (where W is the type size). If $N = 2^n$ and $W = 2^d$, the stride is 2^{n+d} bytes. When $n + d \geq b + s$ (where b is the offset bits and s is the set-index bits), consecutive column elements map to the **same cache set**. Since each set has only w lines, every w accesses cause a **conflict eviction** — the set overflows and useful data is displaced.

0.5.7.2 Padding as a Mitigation **Padding** the matrix — allocating it as $(N + k) \times N$ instead of $N \times N$ — breaks the power-of-two alignment:

$$\text{offset} = (N + k) \times W = 2^{n+d} + k \cdot 2^d \text{ bytes}$$

If $2^{b+s} < 2^{n+d} + k \cdot 2^d$, consecutive column elements now map to **different cache sets**, eliminating the resonance. Even $k = 1$ (adding a single dummy column) can be sufficient.

0.5.8 Enhancing Locality: Data Traversal Strategies

0.5.8.1 Blocking (Tiling) **Loop tiling** (also called **blocking**) is the technique of processing a matrix not row-by-row or column-by-column, but in small rectangular **blocks** (tiles) that fit entirely in the cache.

For matrix-matrix multiplication $C = A \times B$, the naïve implementation:

```

for (int i = 0; i < m; i++)
    for (int k = 0; k < p; k++)
        for (int j = 0; j < n; j++)
            C[i][k] += A[i][j] * B[j][k];

```

suffers from poor locality in the access to B : for each element of C , the entire column of B is traversed, causing approximately mnp cache misses.

Loop interchange (swapping the two inner loops) improves locality for B :

```

for (int i = 0; i < m; i++)
    for (int j = 0; j < n; j++)
        for (int k = 0; k < p; k++)
            C[i][k] += A[i][j] * B[j][k];

```

This reduces expected misses to $\sim 3mnp/L$, saving approximately $mnp(L-2)/L$ misses compared to the naïve nesting.

Tiling goes further by processing the matrices in blocks of size $l \times h \times g$ that fit in L1:

```

for (int ii = 0; ii < m; ii += bsize)
    for (int kk = 0; kk < p; kk += bsize)
        for (int jj = 0; jj < n; jj += bsize)
            for (int i = ii; i < min(ii+bsize, m); i++)
                for (int j = jj; j < min(jj+bsize, n); j++)
                    for (int k = kk; k < min(kk+bsize, p); k++)
                        C[i][k] += A[i][j] * B[j][k];

```

The block size is chosen so that three blocks (from A , B , and C) simultaneously fit in L1 cache. This reduces the cache miss count by a factor proportional to $L/(l \times h \times g)$, which for typical values reaches approximately 0.001 — a thousandfold improvement.

0.5.8.2 The Z-Order (Morton Curve) The **Morton Z-order** curve provides a **cache-oblivious** traversal pattern — one that provides good locality without needing to know the cache size. It works by **interleaving the bits** of the spatial coordinates to produce a 1D index.

For a 2D point (x, y) with binary representations $x = x_i \dots x_1 x_0$ and $y = y_i \dots y_1 y_0$, the Morton index is:

$$Z(x, y) = (y_i x_i \dots y_1 x_1 y_0 x_0)_2$$

This produces the characteristic Z-shaped recursive pattern:

(x, y)	Z
(0,0)	0
(1,0)	1

(x, y)	Z
(0, 1)	2
(1, 1)	3
(2, 0)	4
(3, 0)	5
(2, 1)	6
(3, 1)	7

The Z-curve is one of the **space-filling curves** (Peano curves), which map the 2D plane to a 1D line while approximately preserving spatial proximity. Points that are close in 2D tend to have close Z-indices, meaning they will be stored close together in memory.

Advantage over explicit tiling: the Z-order requires no tuning of block sizes. It delivers good (though not necessarily optimal) performance across all cache sizes — a **cache-oblivious** property.

0.5.8.3 The Hilbert Curve The **Hilbert curve** is another space-filling curve with superior locality-preservation properties compared to the Z-curve. Unlike the Z-curve, the Hilbert curve has **no jumps** — consecutive points along the curve are always spatially adjacent. This means the maximum distance distortion between the 2D space and the 1D memory layout is smaller than for the Z-curve.

Space-filling curves can be used for two distinct purposes:

1. **Determining traversal order** for structured data (e.g., matrix elements)
2. **Reordering unstructured data** in memory: compute each data point's 1D index along the curve, then sort the data by that index so that spatially close points become contiguous in memory

0.5.9 Hot and Cold Fields

0.5.9.1 The Problem Consider a linked-list node with heterogeneous fields:

```
struct my_node {
    double key;
    char my_data[300];
    my_node *next_node;
};
```

When traversing the list to search for a key:

```
while (p != NULL) {
    if (p->key == key) { do_something(); break; }
    p = p->next_node;
}
```

The fields `key` and `next_node` are accessed together at every iteration — they are **temporally local** (the “hot” fields). The 300-byte `my_data` field is accessed only once, after a successful search — it is “cold.” However, `my_data` sits **between** the hot fields, separating them by 300 bytes, which guarantees they will reside on different cache lines.

0.5.9.2 Solution 1: Reorder Fields

```
struct my_node {
    double key;
    my_node *next_node;
    char my_data[300];
};
```

Now `key` and `next_node` are contiguous and fit within a single 64-byte cache line ($8 + 8 = 16$ bytes). The cold data follows, occupying separate lines that are loaded only when needed.

0.5.9.3 Solution 2: Separate Hot and Cold Data A more aggressive restructuring **splits** the node into two structures:

```
struct my_node {
    double key;
    my_node *next_node;
    data_t *my_data;    // pointer to cold data
};

struct data_t {
    char data[300];
};
```

The hot part of each node is now only 24 bytes (`key` + next pointer + data pointer), well within a single cache line. For $N = 10^8$ nodes, the hot working set is $24 \times 10^8 = 2.4$ GB instead of approximately 32 GB — a $13\times$ reduction in the memory footprint that determines traversal performance.

0.5.9.4 General Principle

“What is used together, stays together.”

Fields that are temporally co-accessed (hot fields) should be placed contiguously in memory. Cold fields — accessed only in rare or terminal code paths — should be separated, ideally into a distinct allocation accessed via pointer indirection.

0.5.9.5 Arrays vs. Linked Lists In HPC, **arrays** should be strongly preferred over linked lists for storing collections of homogeneous elements. Arrays provide contiguous memory layout, predictable access patterns amenable to hardware prefetching, and vectorizable traversal. When distinct field types exist, the **Structure of Arrays (SoA)** layout provides the best cache behavior:

```
keys:    [key , key , key , ...]        // contiguous
data_ptrs: [*data , *data , *data , ...] // contiguous
data:    [data , data , ...]            // accessed only on demand
```

0.6 Loop Characterization and Optimization

0.6.1 Arithmetic Intensity

Arithmetic intensity is the ratio of the number of floating-point operations performed to the amount of data moved from memory. It is the primary determinant of whether a loop is **compute-bound** or **memory-bound**, and it dictates the optimization potential.

Class	Operations / Data	Examples	Optimization Potential
$O(N)/O(N)$	Linear ops on linear data	Array scanning, 1D vector ops, scalar products	Limited — inevitably memory-bound for large N
$O(N^2)/O(N^2)$	Quadratic ops on quadratic data	Matrix–vector multiplication, matrix transpose, matrix addition	Moderate — some opportunities for data reuse
$O(N^3)/O(N^2)$	Cubic ops on quadratic data	Matrix–matrix multiplication, dense matrix factorization	Significant — high data reuse enables near-peak performance

0.6.2 $O(N)/O(N)$ Loops: Loop Fusion

For linear-complexity loops, the primary optimization technique (beyond the basic principles already discussed) is **loop fusion**: merging separate loops that traverse the same data into a single loop to avoid redundant memory accesses.

Before fusion:

```
for (int i = 0; i < N; i++)
    A[i] = B[i] * C[i];
for (int i = 0; i < N; i++)
    Q[i] = B[i] + D[i];
```

Array B is loaded from memory twice — once in each loop.

After fusion:

```
for (int i = 0; i < N; i++) {
    A[i] = B[i] * C[i];
    Q[i] = B[i] + D[i];
}
```

Now B[i] is loaded once, used twice. This halves the memory traffic for B.

0.6.3 $O(N^2)/O(N^2)$ Loops: Matrix–Vector Multiplication

Consider matrix–vector multiplication $\mathbf{c} = \mathbf{A}\mathbf{b}$:

```
for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
        C[i] += A[i][j] * B[j];
```

Step 1 — Eliminate redundant memory references:

```
for (int i = 0; i < N; i++) {
    double c_temp = C[i];
    for (int j = 0; j < N; j++)
        c_temp += A[i][j] * B[j];
    C[i] = c_temp;
}
```

This reduces memory accesses from $3N^2$ to $2N^2 + N$.

Step 2 — Unroll and jam:

Unroll the outer loop by a factor m and fuse the inner loops:

```
for (int i = 0; i < N; i += m)
    for (int j = 0; j < N; j++) {
        double b_temp = B[j];
        C[i] += A[i][j] * b_temp;
        C[i+1] += A[i+1][j] * b_temp;
        // ...
        C[i+m-1] += A[i+m-1][j] * b_temp;
    }
```

Now B[j] is loaded once and reused m times. Memory accesses reduce to approximately $N^2(1 + 1/m) + N$.

0.6.4 $O(N^3)/O(N^2)$ Loops: Matrix–Matrix Multiplication

These algorithms have the highest optimization potential. With $2mnp$ floating-point operations on $O(N^2)$ data, there is enormous scope for data reuse. The combination of **tiling**, **unroll-and-jam**, **vectorization**, and **pipeline-aware operation ordering** can push performance very close to theoretical peak. Highly optimized libraries (BLAS implementations such as **OpenBLAS**, **Intel MKL**, **BLIS**) implement these techniques and should generally be used instead of hand-written code.

0.6.5 Loop Unrolling: Detailed Analysis

Loop unrolling is a fundamental code transformation that:

- Reduces loop overhead (counter updates, branch instructions)
- Exposes the **critical data path** and inter-iteration dependencies
- Enables greater **Instruction-Level Parallelism (ILP)**, especially in the presence of memory aliasing

0.6.5.1 Notation Loop unrolling is described using the notation $n \times m$, where:

- n = number of iterations processed per unrolled iteration (the loop counter increments by n)
- m = number of independent **accumulators** used

0.6.5.2 Handling Remainders When the trip count N is not a multiple of the unrolling factor U , a **cleanup loop** handles the residual iterations:

```
int N_ = (N / U) * U;  // largest multiple of U  N
for (int i = 0; i < N_; i += U)
    /* unrolled body */;
for (int i = N_; i < N; i++)
    /* scalar body */;
```

0.6.5.3 Why the Compiler Cannot Unroll Floating-Point Reductions Consider a simple reduction:

```
for (int i = 0; i < N; i++)
    S += A[i];
```

When S and A are `int`, the compiler at `-O3 -march=native` aggressively vectorizes the loop, achieving dramatic speedups. When they are `double`, the compiler **does not** vectorize or reorder, because:

- Floating-point addition is **not associative**
- Changing the order of summation may change the numerical result
- The compiler cannot prove that the programmer's chosen order is not intentional

The programmer must therefore perform the unrolling manually.

0.6.5.4 Step 1: Unrolling 2×1

```
for (int i = 0; i < N - 2; i += 2)
    S = (S + A[i]) + A[i+1];
```

This processes two elements per iteration but still has a single accumulator (S), creating a **loop-carried dependency chain**: each iteration must wait for the previous addition to complete before starting the next.

0.6.5.5 Step 2: Unrolling 2×1 with Reassociation

```
for (int i = 0; i < N - 2; i += 2)
    S = S + (A[i] + A[i+1]);
```

By reassociating the parentheses, the programmer tells the compiler that $A[i] + A[i+1]$ can be computed **independently** of S. The compiler can now:

1. Load 8 doubles at once into a 256-bit YMM register
2. Perform pairwise additions within the register
3. Reduce the critical path length

This subtle change — moving parentheses — can yield significant speedups because it grants the compiler freedom to exploit SIMD and ILP.

0.6.5.6 Step 3: Unrolling 2×2 (Multiple Accumulators)

```
double s0 = 0, s1 = 0;
for (int i = 0; i < N - 2; i += 2) {
    s0 += A[i];
    s1 += A[i+1];
}
S = s0 + s1;
```

Two independent accumulators break the loop-carried dependency entirely: `s0` and `s1` can be updated in **parallel** on separate functional units. The critical path is halved. The compiler, given this structure, can map each accumulator to a separate SIMD register and exploit full pipeline throughput.

0.6.5.7 Higher Unrolling Factors The same principle extends to 4×4 or even 8×8 unrolling. Explicit vectorization (using SIMD intrinsics) can push performance further. Typical unrolling factors range from 2 to 16; beyond that, diminishing returns set in due to register pressure and instruction cache effects.

0.7 Prefetching

0.7.1 The Latency Hiding Problem

The latency gap between CPU operations ($\sim 0.25\text{--}1$ ns) and DRAM access ($\sim 50\text{--}100$ ns, i.e., 150–300 clock cycles at 3–4 GHz) means that a cache miss stalls the pipeline for hundreds of cycles. **Out-of-order execution** can hide some latency by finding independent instructions to execute during the stall, but the **reorder buffer (ROB)** is finite (320–512 entries on modern cores). A sustained stream of cache misses exhausts the ROB, leaving the core idle.

0.7.2 The Prefetching Principle

Prefetching issues memory requests **before** the data is actually needed, so that by the time the load instruction executes, the data is already in cache. The latency is not reduced — DRAM still takes 200 cycles to respond — but it is **overlapped** with computation, effectively making it invisible.

A prefetch is a **fire-and-forget** memory read: it populates a cache line without stalling the issuing instruction.

0.7.3 Prefetch Distance

The critical parameter is the **prefetch distance** D : the number of iterations of look-ahead needed to fully hide the memory latency. For a loop with body cost T cycles per iteration and DRAM latency L cycles:

$$D \approx \frac{L}{T}$$

For a simple arithmetic loop with $T \approx 4$ cycles and $L \approx 200$ cycles, $D \approx 50$ iterations of look-ahead. Cheap loops require large D (making prefetching fragile); expensive loops require small D (making prefetching easier).

0.7.4 Three Levels of Prefetching

0.7.4.1 1. Hardware Prefetcher A dedicated CPU unit that **monitors cache miss addresses** and detects regular patterns (sequential scans, constant strides). It proactively loads predicted cache lines.

Strength	Weakness
Remarkably effective for regular access patterns	Can only recognize patterns it was designed for
Self-tuning: dynamically measures T and L	Requires observing several misses before prediction begins
Zero programmer effort	Fails completely for irregular or data-dependent patterns

0.7.4.2 2. Software Prefetcher Explicit prefetch instructions inserted by the programmer or compiler:

```
__builtin_prefetch(&data[address], 0, 0);
// Arguments: address, read(0)/write(1), locality hint (0=NTA, 3=L1)
```

Strength	Weakness
Can handle irregular access patterns	Placement is system-dependent and fragile
Programmer can compute future addresses	“Too early” → eviction before use; “too late” → incomplete hiding

0.7.4.3 3. Out-of-Order Execution as Implicit Prefetcher The OoO engine can issue **multiple independent loads** simultaneously, creating **Memory-Level Parallelism (MLP)**: several DRAM requests are in flight at once, and their latencies overlap. Code that exposes high MLP — through loop unrolling, ILP-friendly structure, and absence of false dependencies — benefits from implicit prefetching without any explicit instructions.

The binding constraint is often not the ROB size but the number of **Miss Status Holding Registers (MSHRs)** — typically 10–20 per core — which limit how many outstanding cache misses the CPU can track simultaneously.

0.7.5 Practical Example: Binary Search with Prefetching

A standard binary search exhibits an irregular access pattern — each iteration accesses a location determined by the comparison result of the previous iteration, making it impossible for the hardware prefetcher to predict.

Solution: speculatively prefetch **both** possible next midpoints before the comparison result is known:

```
while (low <= high) {
    mid = (low + high) / 2;
    __builtin_prefetch(&data[(mid + 1 + high) / 2], 0, 0); // right branch
    __builtin_prefetch(&data[(low + mid - 1) / 2], 0, 0);  // left branch
    if (data[mid] < Key)
        low = mid + 1;
    else if (data[mid] > Key)
```

```

        high = mid - 1;
    else
        return mid;
}

```

One of the two prefetches is always “wasted” (fetching data for the branch not taken), but the other ensures the next comparison’s data is already in cache. Experimental results show a **1.64× speedup** (e.g., 20.75 s → 12.62 s). Analysis of the **Line Fill Buffer (LFB)** hit rate reveals that most prefetches arrive **late** (data is in transit but not yet in cache when the load executes — ~68% LFB hits) rather than **perfectly** (~27% L1 hits), indicating that further tuning of the prefetch distance could yield additional gains.

0.7.6 Prefetching by Pre-loading

An alternative to explicit prefetch instructions is **pre-loading**: loading the next iteration’s data into a register at the beginning of the current iteration, giving the memory subsystem time to deliver it while the current iteration’s computation proceeds:

```

elem a = elem[0];
for (int i = 0; i < 4*N_4; i += 4) {
    elem e = elem[i + 4]; // non-blocking miss - future data
    elem b = elem[i + 1]; // likely cache hit (same line as elem[i])
    elem c = elem[i + 2]; // likely cache hit
    elem d = elem[i + 3]; // likely cache hit
    Elaborate(a);
    Elaborate(b);
    Elaborate(c);
    Elaborate(d);
    a = e;                // by now, e should be in cache
}

```

This technique is less fragile than explicit prefetch directives because the compiler is free to schedule the load optimally. However, with `-O3`, the compiler already attempts to place loads as early as possible relative to their use, potentially making manual pre-loading redundant.

0.7.7 Best Practices Summary

For **regular access patterns**: the hardware prefetcher is highly effective. The programmer’s role is to ensure regularity (sequential or constant-stride access), unroll to exploit ILP, and if possible, vectorize explicitly.

For **irregular access patterns**: explicit prefetching or pre-loading can be beneficial, but results are system-dependent and require careful profiling.

0.8 Branches: Control Flow and Performance

0.8.1 Conditional Execution Mechanisms

Modern architectures provide two distinct low-level mechanisms for conditional execution:

1. **Conditional control flow** — modifying the sequence of executed instructions (jump/branch instructions)
2. **Conditional data flow** — modifying which data is moved or computed, without altering the instruction sequence (conditional move, `cmov`)

0.8.2 Conditional Control Flow: Jump Instructions

At the machine level, altering the execution flow requires a **jump instruction**. Jumps can be:

- **Unconditional** (`jmp`): always transfers control to the specified target
- **Conditional** (`je`, `jne`, `jl`, `jg`, `js`, ...): transfers control only if a specific condition, encoded in the **flags register**, is satisfied

The flags register is updated after every arithmetic or logical operation:

Flag	Meaning
CF (Carry Flag)	A carry out of the most significant bit occurred (unsigned overflow)
ZF (Zero Flag)	The most recent operation produced a zero result
SF (Sign Flag)	The most recent operation produced a negative result
OF (Overflow Flag)	A two's-complement overflow occurred

0.8.2.1 Assembly Examples **For loop** — the loop body executes repeatedly via a backward conditional jump:

```
.L2:
    ; loop body (e.g., modify array element via [rax])
    add rax, 8           ; increment pointer (equivalent to i++)
    cmp rax, rbp         ; compare with termination value
    jne .L2              ; jump back if not equal (i.e., if i < N)
```

If statement — the **true** branch is placed immediately after the comparison (fall-through), while the **false** branch is reached via a conditional jump:

```
mov    eax, [a]
cmp    eax, [b]          ; computes a - b, sets flags
```

```

    jle .false_branch    ; jump to false branch if a <= b
    ; true branch code
    jmp .end             ; unconditional jump past false branch
.false_branch:
    ; false branch code
.end:

```

Code layout principle: the compiler places the expected-true branch as the fall-through path (no jump) and the expected-false branch as the jump target. When coding, the programmer should place the most likely outcome in the `if` clause (not the `else`) to preserve code locality. Explicit hints can be given:

```

if (__builtin_expect(condition, 1)) { /* likely path */ } // C (GCC)
if (condition) [[likely]] { /* likely path */ }           // C++20

```

0.8.3 Conditional Data Flow: The `cmov` Instruction

The **conditional move** (`cmov`) instruction precomputes **both** outcomes of a conditional expression and selects the correct one based on the condition — **without issuing any jump instruction**:

```

mov ecx, [a_plus_b]    ; result if a > b
mov edx, [a_minus_b]   ; result if a < b
cmp ebx, eax           ; compare a and b
cmovg edx, ecx         ; if a > b, overwrite edx with ecx

```

The execution flow is **perfectly linear** and fully predictable — no branch is taken, no pipeline flush is possible.

Limitation: `cmov` is applicable only when:

- Both branch outcomes produce a **single scalar value**
- Neither branch has **side effects** (no memory writes, no function calls, no potential faults)
- Neither branch would cause a **cache miss** that the conditional branch would have avoided (if one branch rarely executes, forcing its computation via `cmov` may trigger unnecessary cache misses)

0.8.4 The Cost of Branch Misprediction

Modern CPUs achieve high throughput through deep pipelines (10–20 stages) and out-of-order execution with hundreds of instructions in flight simultaneously (ROB sizes of 320–512). The **branch predictor** — a sophisticated hardware unit — guesses whether each conditional jump will be taken, allowing the pipeline to continue speculatively.

When the prediction is **correct** (95–99% of the time for well-structured code), the pipeline operates at full throughput. When it is **wrong** — a **branch misprediction** — all speculatively executed instructions must be discarded and the pipeline refilled from the correct path. The penalty is **10–30 cycles** per misprediction.

Quantitative impact example: with 140 instructions in flight and 1 branch every 7 instructions (20 branches), even a 95% prediction accuracy yields only a $(0.95)^{20} \approx 36\%$ probability that the pipeline is never flushed. At 90% accuracy, this drops to $(0.90)^{20} \approx 12\%$.

0.8.5 Strategies for Minimizing Branch Impact

0.8.5.1 Strategy 1: Loop Hoisting of Conditionals When conditional tests inside a loop are **loop-invariant** (do not depend on the iteration variable), they should be moved outside the loop, creating separate specialized loops for each case:

```
// Before: if-forest inside loop
for (i = 0; i < top; i++) {
    if (case1 == 0) {
        if (case2 == 0) result += i;
        else           result *= i;
    } else {
        if (case2 == 0) result += log10((double)i);
        else           result *= sin((double)i);
    }
}

// After: decisions moved outside
if (case1 == 0 && case2 == 0)
    for (i = 0; i < top; i++) result += i;
else if (case1 == 0 && case2 != 0)
    for (i = 0; i < top; i++) result *= i;
// ... etc.
```

Compilers handle loop-invariant **code** hoisting well, but hoisting **control flow** is a more restricted transformation — manual restructuring is often necessary.

0.8.5.2 Strategy 2: Branchless Arithmetic (Predication) For data-dependent conditionals with simple scalar outcomes, replace the branch with arithmetic:

```
// Branch version (unpredictable):
if (data[i] > PIVOT)
    sum += data[i];

// Branchless version:
sum += (data[i] > PIVOT) * data[i];

// Or equivalently:
sum += (data[i] > PIVOT) ? data[i] : 0;
```

At -O3, the compiler emits `cmov` or SIMD masked operations. The branchless version produces **identical performance regardless of data predictability**, while the branch version suffers severely with random data.

When to use: branchless code wins when the branch is **unpredictable**; it loses when the branch is highly predictable and one outcome is significantly cheaper than the other.

0.8.5.3 Strategy 3: Data Partitioning If all data is available in advance, **partition** the data into separate arrays by category, then run specialized, branchless, vectorizable loops over each partition:

```
// Instead of: if (data[i] belongs to category A) ... else ...  
// Partition first, then:  
for (i = 0; i < n_A; i++) process_A(data_A[i]);  
for (i = 0; i < n_B; i++) process_B(data_B[i]);
```

The per-partition loops are branchless and vectorizable. The overhead is the partitioning step, but for large datasets with expensive per-element operations, this is typically a net win.

0.8.5.4 Strategy 4: Restructure Data Layout (AoS $\rightarrow$ SoA) If a conditional discriminates element categories stored in an **Array of Structures (AoS)**, converting to a **Structure of Arrays (SoA)** can eliminate the conditional entirely by separating the categories into distinct arrays.

0.8.5.5 Strategy 5: Function Pointers and Lookup Tables For many discrete cases, a **switch statement** is compiled into a **jump table** — an array of code addresses indexed by the case value:

```
// Compiled as:  
table_of_addr[N] = { addr_case_0, addr_case_1, ..., addr_default };  
if (case > N) jump to addr_default;  
jump to table_of_addr[case];
```

This replaces a chain of conditional branches with a single **indirect branch**. The indirect branch predictor can learn regular patterns, but for random or data-dependent case values, the **Indirect Branch Target Buffer** is constantly evicted, and prediction fails.

Function pointers offer similar behavior but carry additional overhead from the function call itself and the lost optimization opportunities (no inlining, no vectorization across the call boundary).

0.8.5.6 Strategy 6: Algorithm Redesign When an if-forest is deeply nested, data-dependent, and not reducible to simple arithmetic, it is typically a **symptom** of an algorithm that mixes heterogeneous computations in a single traversal. The data should be restructured so that homogeneous subsets are processed in separate, specialized, branchless loops.

0.8.6 Profile-Guided Optimization for Branches

The compiler can collect **real branch statistics** from representative runs and use them to optimize branch prediction hints and code layout:

```
gcc -fprofile-generate -O3 source.c -o instrumented
./instrumented                # run with representative input
gcc -fprofile-use -O3 source.c -o optimized
```

This is particularly effective for complex code with many branches whose probabilities cannot be determined statically.

0.8.7 A Word of Caution

It is easy to become absorbed in micro-optimization — hunting single instructions and obsessing over pipeline behavior. The most impactful optimization decisions remain, by far:

1. **The algorithm chosen**
2. **The data model designed**
3. **The overall quality, cleanness, and robustness of the code**

Micro-optimization is powerful but should be applied only after these foundational aspects are settled, and only to **profiled hot spots** where the payoff justifies the investment of programmer time.